MedWarehouse

Event-Driven Inventory Analytics Warehouse
Technical Documentation

Architecture • Developer Guide • API Reference • Data Dictionary
Deployment • Security • Runbook • Domain Specifications

Complete Technical Reference

MedWarehouse — Technical Documentation

- Medical Warehouse — Event-Driven Inventory Analytics
  - Table of Contents
  - Overview
  - Architecture
  - Data Flow
  - Repository Layout
  - Tech Stack
  - Infrastructure
  - Configuration
  - Quick Start
  - CLI Reference
  - Operator Webapp
  - Monitoring Platform
  - Warehouse Schema
  - Security Model
  - Airflow DAGs
  - Tests
- Architecture
  - System Overview
  - Layer Map
  - Kafka Domain Pipelines
  - Analytical Domains (FDW-backed)
  - Security Model
  - Package Structure
  - Orchestration
- Developer Guide
  - Table of Contents
  - 1. System Purpose and Context
  - 2. High-Level Architecture
  - 3. Repository Layout
  - 4. Configuration System
  - 5. Data Flow: End-to-End
  - 6. Layer 1 — Event Producers
  - 7. Layer 2 — Apache Kafka
  - 8. Layer 3 — Spark Bronze and Silver Pipelines
  - 9. Layer 4 — PostgreSQL Analytical Warehouse
  - 10. Layer 5 — Monitoring Platform and Operator Webapp
  - 11. The Three Business Domains
  - 12. Nine Analytical Domains in the Database
  - 13. The Control Plane
  - 14. Alert System
  - 15. Airflow Orchestration
  - 16. Infrastructure and Docker Compose
  - 17. Testing Strategy
  - 18. Key Design Decisions and Trade-offs
  - 19. Important Nuances and Gotchas
  - 20. Common Developer Workflows
  - 21. Extending the System: Adding a New Domain

- Deployment Guide
  - Prerequisites
  - Environment Configuration
  - Infrastructure Startup
  - Python Environment Setup
  - Warehouse Bootstrap
  - Running the Inventory Pipeline (Step-by-Step)
  - Running the Procurement Pipeline
  - Running the Sales Pipeline
  - Starting the Operator Webapp
  - Starting the Monitoring Platform
  - Running via Airflow
  - Port Reference
  - Environment Variables Reference
  - Upgrading
  - Health Checks
- Development Guide
  - Repository Structure
  - Quick Setup
  - Running Tests
  - Key Design Patterns
  - Common Development Tasks
- Security Model
  - Authentication
  - Database Role Model
  - Credential Management
  - Sensitive Data Handling
  - Regulatory Compliance
  - Network Security
  - Known Security Gaps (Tracked)
- Operator Webapp — API Reference
  - Health
  - Stock Entry (`/api/v1/stock`)
  - Reference Data (`/api/v1`)
  - Reorder Policies (`/api/v1/reorder-policies`)
  - Purchase Orders (`/api/v1/orders`)
  - Monitoring Platform API Reference
- Runbook
  - Quick Start (all three pipelines)
  - Local Environment
  - Inventory Pipeline
  - Procurement Pipeline
  - Sales Pipeline
  - Airflow
  - Bootstrap Replay
  - Quality Checks
  - Reorder Automation
  - Silver Quarantine Investigation
  - Recovery: Bronze Stream Stuck
  - Infrastructure
  - Compliance Checks

- Data Dictionary
  - Source Database — `medwarehouse_master` (OLTP)
  - Analytics Database — `medwarehouse_analytics`
- Domain: Inventory Management
  - Purpose
  - Status
  - Event Types
  - Pipeline
  - Validation Rules (Silver)
  - Key Analytics Objects
  - Quality Checks
  - CLI Commands
  - Airflow DAG
- Domain: Procurement Lifecycle
  - Purpose
  - Status
  - Event Types
  - PO Lifecycle State Machine
  - Pipeline
  - Key Analytics Objects
  - Quality Checks
  - Integration with Reorder System
  - CLI Commands
  - Airflow DAG
- Domain: Sales & Revenue
  - Purpose
  - Status
  - Event Types
  - Pipeline
  - Key Analytics Objects
  - Quality Checks
  - Power BI Usage
  - CLI Commands
  - Airflow DAG
- Domain: Compliance & Audit
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - Alert Rules
  - Regulatory Obligations
  - Security Use Cases
  - Power BI Usage
- Domain: Supplier Management
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - Alert Rules
  - Procurement Performance
  - Power BI Usage

- Domain: Financial Analytics
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - GST / Tax Analytics
  - Power BI Usage
  - Note on Completeness
- Domain: Customer Analytics
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - Privacy Considerations
  - Segmentation
  - Power BI Usage
- Domain: Operations & Staff Performance
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - Operational Anomaly Patterns
  - Power BI Usage
  - Privacy Note
- Domain: Prescription Analytics & Compliance
  - Purpose
  - Status
  - Source Tables (via FDW)
  - Key Analytics Objects
  - Alert Rule
  - Regulatory Context
  - Power BI Usage
  - No Pipeline Required
- Functional Requirements Checklist
  - Status legend
  - Summary
  - Functional Checklist
  - Remaining Work
- Control Panel Testing Plan
  - Scope
  - Test objectives
  - Test environments
  - Functional requirements
  - Non-functional requirements
  - Non-functional test checklist
  - Existing automated coverage
  - Recommended automation backlog
  - Suggested release gate for the control panel
  - Overall assessment

Medical Warehouse — Event-Driven Inventory Analytics

Event-driven inventory analytics warehouse for a medical/pharmaceutical operation. Operator-submitted stock movement events travel through Apache Kafka into a Bronze → Silver → Gold Spark pipeline, land in a queryable analytical warehouse in PostgreSQL, and are served to Power BI and a REST API consumed by warehouse staff.

Table of Contents

- Overview
- Architecture
- Data Flow
- Repository Layout
- Tech Stack
- Infrastructure
- Configuration
- Quick Start
- CLI Reference
- Operator Webapp
- Monitoring Platform
- Warehouse Schema
- Security Model
- Airflow DAGs
- Tests

Overview

The system tracks four inventory event types — `STOCK_RECEIVED`, `STOCK_SOLD`, `STOCK_ADJUSTED`, `STOCK_EXPIRED` — alongside procurement and sales events, across three independently-orchestrated Kafka → Parquet → PostgreSQL pipelines.

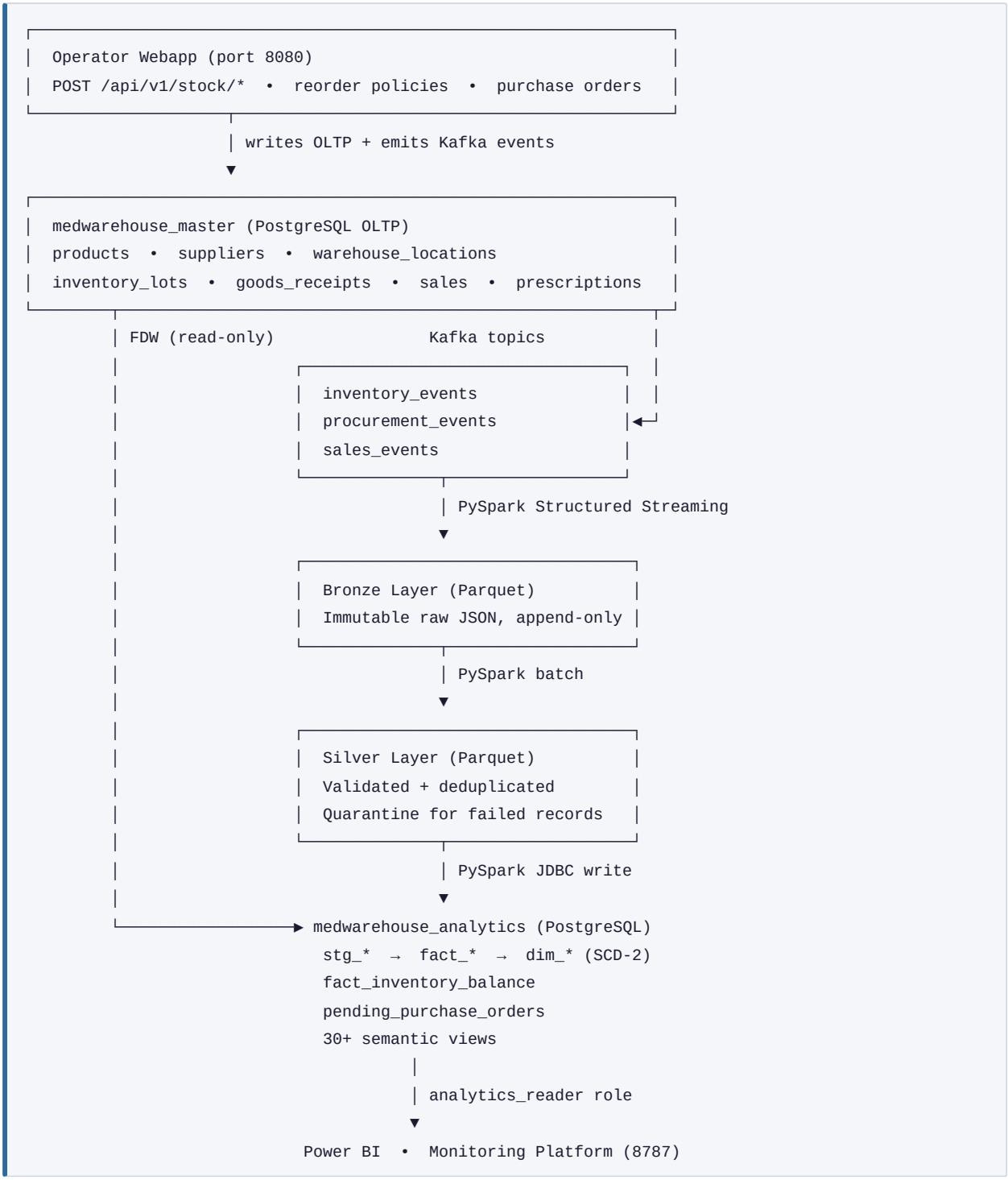
Inventory quantities are **never stored as a running balance**. Every stock movement is appended as an immutable event; the current balance is derived by summing all deltas for a given product–warehouse–batch combination. A physical snapshot table (`fact_inventory_balance`) is maintained after each pipeline run so Power BI can avoid re-summing millions of events on each query.

Nine analytical domains are served through semantic views in PostgreSQL: inventory, procurement, sales, prescriptions/compliance, supplier management, financial (AP/AR), customer analytics, audit, and staff performance.

Two independent Flask services complement the data pipeline:

Service	Port	Audience	Purpose
Operator Webapp	8080	Warehouse staff	Stock entry, purchase order management, reorder policy configuration
Monitoring Platform	8787	Engineers / DevOps	Operational dashboard, pipeline health probes, job runner

Architecture



Data Flow

Each step maps to a CLI command or an Airflow DAG task:

Step	Command	What it does
1	`produce inventory`	Emit sample events to the `inventory_events` Kafka topic
2	`spark inventory-bronze`	Structured Streaming: Kafka → Bronze Parquet (long-running)
3	`spark inventory-silver`	Batch: Bronze → validated Silver Parquet + Quarantine
4	`spark stage-inventory-events`	Batch JDBC write: Silver → `analytics.stg_inventory_events`
5	`warehouse refresh-dimensions`	SCD-2 upsert from FDW-backed `master.*` dimension tables
6	`warehouse load-facts`	Idempotent insert from staging → `fact_inventory_events`
7	`warehouse refresh-views`	Recreate all semantic BI views
8	`warehouse quality-checks`	Seven integrity assertions; raises on any failure
9	`warehouse refresh-balance`	MERGE `v_inventory_balance` → `fact_inventory_balance` snapshot
10	`warehouse check-reorders`	Insert `DRAFT` purchase orders for stock below reorder threshold

Steps 4–10 are composed into `orchestration build-gold` and into the Airflow DAG `inventory_gold_pipeline`.

Silver validation rules

The following violations route records to `data/quarantine/<domain>_events/`:

- Top-level fields (`event_id`, `event_type`, `event_time`, `producer`) must be non-null
- `event_type` must be a recognised value for the domain
- `product_id`, `warehouse_id`, `batch_number`, `quantity_delta` must be non-null
- `expiry_date` required for `STOCK_RECEIVED`, `STOCK_SOLD`, `STOCK_EXPIRED`
- `STOCK_RECEIVED` quantity must be positive; `STOCK_SOLD` / `STOCK_EXPIRED` must be negative
- `STOCK_ADJUSTED` must be non-zero and carry a `reason`
- `STOCK_SOLD` must carry a `sale_id`
- Duplicate `event_id` values are quarantined (lowest-offset record wins)

Quality checks

Seven assertions run at the end of every Gold build:

Check	What it catches
`staging_null_business_keys`	Staging rows missing `product_id`, `warehouse_id`, or `batch_number`
`staging_duplicate_event_ids`	Duplicate `event_id` values still in staging
`staged_events_not_loaded`	Staging rows that did not make it into the fact table
`fact_null_dimension_keys`	Fact rows with null `product_sk` or `warehouse_sk`
`fact_orphan_product_dimension`	Fact rows whose `product_sk` no longer exists in `dim_product`
`fact_orphan_warehouse_dimension`	Fact rows whose `warehouse_sk` no longer exists in `dim_warehouse`
`fact_rows_in_default_partition`	Fact rows that fell into the catch-all partition (missing time-range partition)

Repository Layout

```
.
├── src/
│   ├── medwarehouse/                  # Core pipeline package (python -m medwarehouse)
│   │   ├── __main__.py                # Entry point
│   │   ├── cli.py                     # CLI argument parser and command dispatch
│   │   ├── config.py                  # All settings from env vars (lru_cache singleton)
│   │   ├── logging.py                 # Structured logging setup
│   │   ├── contracts/
│   │   │   ├── _utils.py              # Shared: deterministic_uuid(), to_utc_iso()
│   │   │   ├── inventory.py           # STOCK_* event schema + validator + sample generator
│   │   │   ├── procurement.py         # PO_* event schema + validator
│   │   │   └── sales.py               # SALE_* event schema + validator
│   │   ├── producers/
│   │   │   ├── common.py              # emit_events() + run_producer() shared utilities
│   │   │   ├── inventory.py           # Inventory sample producer
│   │   │   ├── procurement.py         # Procurement sample producer
│   │   │   └── sales.py               # Sales sample producer
│   │   ├── spark/
│   │   │   ├── session.py             # SparkSession builder
│   │   │   └── jobs/
│   │   │       ├── _bronze.py         # Shared: Kafka → Parquet streaming logic
│   │   │       ├── _silver.py         # Shared: validate + deduplicate + quarantine
│   │   │       ├── _stage.py          # Shared: Silver → PostgreSQL JDBC write
│   │   │       ├── inventory_bronze.py / _silver.py / _stage.py
│   │   │       ├── procurement_bronze.py / _silver.py / _stage.py
│   │   │       └── sales_bronze.py / _silver.py / _stage.py
│   │   ├── warehouse/
│   │   │   ├── _common.py             # Shared: validate_silver_ready(),
call_warehouse_function()
│   │   │   ├── postgres.py            # psycopg2 connection context manager
│   │   │   ├── sql_runner.py          # Template-variable SQL execution
│   │   │   ├── inventory.py           # Python wrappers for inventory stored procedures
│   │   │   ├── procurement.py         # Python wrappers for procurement stored procedures
│   │   │   └── sales.py               # Python wrappers for sales stored procedures
│   │   └── platform/                  # Monitoring dashboard (Flask, port 8787)
│   │       ├── api/                   # Blueprint: REST API + HTML page routes
│   │       ├── auth.py                # API key middleware (hmac.compare_digest)
│   │       ├── cache.py               # TTLCache with stampede protection
│   │       ├── control/               # Job runner, infra actions, job catalog (SQLite-backed)
│   │       ├── probes/                # Health probes (infra, warehouse, pipeline, artifacts,
jobs, airflow)
│   │       ├── services/              # StatusService, AlertEngine
│   │       └── ui/                    # Jinja2 templates + static assets
│   └── medwarehouse_webapp/           # Operator data-entry webapp (Flask, port 8080)
│       ├── __main__.py                # Entry point: python -m medwarehouse_webapp
│       ├── app.py                     # Flask application factory
│       ├── db.py                      # DB connection helpers
│       ├── api/
│       │   ├── stock.py               # Blueprint: stock movement endpoints
│       │   ├── inventory.py           # Blueprint: reference data reads
│       │   ├── reorder.py             # Blueprint: reorder policy CRUD
│       │   └── orders.py              # Blueprint: purchase order lifecycle
│       └── services/
│           ├── stock_service.py       # Validate → write OLTP → emit Kafka → trigger pipeline
│           ├── reorder_service.py     # Reorder policy CRUD against analytics DB
│           ├── order_service.py       # PO state machine + supplier email dispatch
│           └── idempotency.py         # In-memory idempotency key store with background TTL
```

```
eviction
├── sql/warehouse/                      # SQL bootstrap files; executed in numeric order
│   ├── 01_roles_and_schemas.sql        # Roles, schemas, postgres_fdw extension
│   ├── 02_fdw_master_access.sql        # Foreign server + user mappings (16 tables)
│   ├── 03_inventory_dimensions.sql     # SCD-2 dimension tables + refresh functions
│   ├── 04_inventory_fact_and_staging.sql
│   ├── 05_inventory_loads_and_views.sql
│   ├── 06_reorder_and_balance.sql      # fact_inventory_balance, reorder_policies,
pending_purchase_orders
│   ├── 07_reorder_functions.sql        # refresh_inventory_balance(), check_reorder_thresholds()
│   ├── 08_procurement_warehouse.sql    # Procurement staging, facts, views, quality checks
│   ├── 09_sales_warehouse.sql          # Sales staging, facts, views + dim_customer
│   └── 10_analytical_domains.sql       # Cross-domain views (compliance, financial, audit, staff)
├── airflow/dags/
│   ├── inventory_gold_pipeline.py      # 8 tasks (includes balance refresh + reorder check)
│   ├── procurement_gold_pipeline.py    # 6 tasks
│   └── sales_gold_pipeline.py          # 6 tasks
├── schemas/kafka/
│   ├── inventory_events.json
│   ├── procurement_events.json
│   └── sales_events.json
├── docker/
│   ├── airflow/Dockerfile              # Airflow image with PySpark + project dependencies
│   └── postgres/init/                  # DB creation scripts + master data restore
├── scripts/
│   ├── lib/common.sh                   # Shared bash utilities sourced by all scripts
│   ├── start_services.sh               # Start PostgreSQL + Kafka (optionally + Airflow)
│   ├── stop_services.sh
│   ├── setup_local_stack.sh
│   ├── run_inventory_flow.sh           # End-to-end inventory pipeline convenience script
│   ├── run_procurement_flow.sh
│   └── run_sales_flow.sh
├── tests/
│   ├── test_cli.py                     # CLI argument parsing smoke tests
│   ├── test_config.py                  # Settings loading and credential validation
│   ├── test_contracts.py               # Inventory event building and validation rules
│   ├── test_domain_contracts.py        # Procurement and sales contract validation
│   ├── test_platform.py                # StatusService, ControlPlaneStore, Flask routes
│   ├── test_sql_runner.py              # SQL template rendering
│   ├── test_warehouse_functions.py     # Warehouse wrappers, auth middleware, credential warnings
│   └── test_webapp_services.py         # Stock service, IdempotencyStore, TTLCache stampede, order
state machine
├── data/                               # Runtime output (Bronze/Silver/Quarantine Parquet; SQLite
store)
├── docs/                               # Project documentation
├── jars/
│   └── postgresql-42.7.3.jar           # JDBC driver for Spark → PostgreSQL
├── docker-compose.yml
├── pyproject.toml
├── requirements.txt
├── requirements-dev.txt
└── .env.example
```

Tech Stack

Layer	Technology	Version
Language	Python	3.11+
Stream processing	PySpark (Structured Streaming + batch)	3.5.1
Message broker	Apache Kafka (Confluent Platform)	7.5.0
Coordination	Apache ZooKeeper	7.5.0
Analytical store	PostgreSQL with `postgres_fdw`	16
DB driver (Python)	psycopg2-binary	2.9.9
DB driver (Spark)	PostgreSQL JDBC	42.7.3
Kafka client	confluent-kafka	2.4.0
Orchestration	Apache Airflow (LocalExecutor)	2.8.1
Monitoring + Webapp	Flask + Jinja2	2.x
Containerisation	Docker Compose	v2

Infrastructure

Core services (always required):

Service	Image	Port	Role
`postgres`	`postgres:16`	5432	Hosts both `medwarehouse_master` and `medwarehouse_analytics`
`zookeeper`	`confluentinc/cp-zookeeper:7.5.0`	2181	Kafka coordination
`kafka`	`confluentinc/cp-kafka:7.5.0`	9092	Event bus (external); 29092 (internal Docker network)

Optional Airflow services (started with `--with-airflow`):

Service	Port	Role
`airflow-webserver`	**8090**	DAG management UI — remapped from the default 8080 to avoid conflict with the operator webapp
`airflow-scheduler`	—	DAG scheduling backend

All services have `healthcheck` definitions and `restart: unless-stopped`. Kafka and the Airflow application services use `depends_on: condition: service_healthy` so containers do not race on startup.

All services propagate `HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` from `.env` for corporate proxy environments.

Configuration

Copy `.env.example` to `.env`. All application settings are prefixed `MW_`.

Variable	Default	Description
`MW_ENV`	`local`	Environment name; non-`local`/`dev` environments enforce strong credentials
`MW_KAFKA_BOOTSTRAP_SERVERS`	`localhost:9092`	Kafka bootstrap address
`MW_KAFKA_INVENTORY_TOPIC`	`inventory_events`	Inventory Kafka topic name
`MW_MASTER_DB_*`	`localhost:5432/medwarehouse_master`	OLTP source database
`MW_ANALYTICS_ADMIN_DB_*`	`localhost:5432/medwarehouse_analytics`	Analytics DB (admin role)
`MW_ANALYTICS_WRITER_DB_*`	same host / `spark_writer`	Analytics DB (Spark write role)
`MW_ANALYTICS_READER_DB_*`	same host / `analytics_reader`	Analytics DB (BI read role)
`MW_ANALYTICS_FDW_READER_USER`	`fdw_reader`	FDW user inside the analytics DB
`MW_SPARK_MASTER`	`local[*]`	Spark master URL
`MW_PRODUCER_MAX_EVENTS`	`10`	Default event cap for sample producers
`MW_SAMPLE_SEED`	`medical-warehouse-local`	Determinism seed for reproducible sample data
`MW_SAMPLE_START_TIME`	`2026-01-01T08:00:00+00:00`	First event timestamp for sample data
`MW_WEBAPP_API_KEY`	(empty)	Operator webapp API key — empty disables auth (dev only)
`MW_PLATFORM_API_KEY`	(empty)	Monitoring platform API key — empty disables auth (dev only)
`MW_PLATFORM_SECRET_KEY`	(random per-process)	Flask session secret for flash messages
`MW_PLATFORM_HOST`	`127.0.0.1`	Platform host used by the webapp for pipeline trigger calls
`MW_PLATFORM_PORT`	`8787`	Platform port
`MW_SMTP_HOST`	(empty)	SMTP server for supplier PO emails — empty logs instead of sending
`AIRFLOW_ADMIN_PASSWORD`	`admin`	Airflow admin password; set before first `docker compose up`

Quick Start

```
# 1. Copy and configure environment
cp .env.example .env

# 2. Start core infrastructure (PostgreSQL + Kafka)
./scripts/start_services.sh
# To also start Airflow (UI at http://localhost:8090):
./scripts/start_services.sh --with-airflow

# 3. Set Python path
export PYTHONPATH=src

# 4. Bootstrap the analytics warehouse (run once — applies all 10 SQL files)
python -m medwarehouse --log-level INFO warehouse bootstrap

# 5. Start Bronze ingestion in a dedicated terminal (long-running stream)
python -m medwarehouse spark inventory-bronze

# 6. Emit sample inventory events (second terminal)
python -m medwarehouse produce inventory --max-events 20

# 7. Run the full Gold pipeline
python -m medwarehouse spark inventory-silver
python -m medwarehouse orchestration build-gold

# 8. Start the monitoring platform
python -m medwarehouse platform serve --host 127.0.0.1 --port 8787
# Browse to http://127.0.0.1:8787

# 9. Start the operator webapp (separate terminal)
python -m medwarehouse_webapp --host 127.0.0.1 --port 8080
# Health check: curl http://127.0.0.1:8080/health
```

Or use the convenience script for the full inventory pipeline end-to-end:

```
./scripts/run_inventory_flow.sh 20
```

CLI Reference

All commands use the form `python -m medwarehouse [--log-level LEVEL] <group> <subcommand> [options]`.

`produce`

```
python -m medwarehouse produce {inventory|procurement|sales} [--max-events N] [--dry-run]
```

Emits deterministic sample events to the corresponding Kafka topic. The inventory producer cycles `STOCK_RECEIVED → STOCK_ADJUSTED → STOCK_SOLD → STOCK_RECEIVED → STOCK_EXPIRED` with a configurable seed and start time for reproducibility. `--dry-run` logs events without publishing to Kafka.

`spark`

```
python -m medwarehouse spark inventory-bronze [--starting-offsets {earliest|latest}]
python -m medwarehouse spark inventory-silver
python -m medwarehouse spark stage-inventory-events

python -m medwarehouse spark procurement-bronze [--starting-offsets {earliest|latest}]
python -m medwarehouse spark procurement-silver
python -m medwarehouse spark stage-procurement-events

python -m medwarehouse spark sales-bronze [--starting-offsets {earliest|latest}]
python -m medwarehouse spark sales-silver
python -m medwarehouse spark stage-sales-events
```

`*-bronze` jobs are long-running Structured Streaming processes. All others are batch jobs.

`warehouse`

```
python -m medwarehouse warehouse bootstrap
python -m medwarehouse warehouse refresh-dimensions
python -m medwarehouse warehouse load-facts
python -m medwarehouse warehouse refresh-views
python -m medwarehouse warehouse quality-checks
python -m medwarehouse warehouse refresh-balance
python -m medwarehouse warehouse check-reorders
python -m medwarehouse warehouse inventory-gold
```

Subcommand	Description
`bootstrap`	Runs all SQL files in `sql/warehouse/` in numeric order
`refresh-balance`	MERGEs `v_inventory_balance` into the physical `fact_inventory_balance` snapshot
`check-reorders`	Compares `fact_inventory_balance` against `reorder_policies`; inserts `DRAFT` purchase orders
`inventory-gold`	Shortcut: `refresh-dimensions → load-facts → refresh-views → quality-checks`

Procurement and sales equivalents: `refresh-procurement-dimensions`, `load-procurement-facts`, `procurement-quality-checks`, `build-procurement-gold`; and equivalently for sales.

`orchestration`

```
python -m medwarehouse orchestration validate-silver
python -m medwarehouse orchestration stage-events
python -m medwarehouse orchestration build-gold
python -m medwarehouse orchestration build-procurement-gold
python -m medwarehouse orchestration build-sales-gold
```

`build-gold` — full post-Silver inventory pipeline: validate → stage → refresh-dims → load-facts → refresh-views → quality-checks → refresh-balance → check-reorders.

`platform`

```
python -m medwarehouse platform serve [--host HOST] [--port PORT] [--debug]
```

Starts the Flask monitoring platform (default `127.0.0.1:8787`).

Operator Webapp

Entry point: `python -m medwarehouse_webapp [--host HOST] [--port PORT]`

The operator webapp (port 8080) is a REST API for warehouse staff. It shares the master and analytics PostgreSQL databases with the pipeline but runs as a completely independent Flask process.

Stock entry

Each POST endpoint: validates the request body → writes to `inventory_lots` in the OLTP database → emits a Kafka event → fires an async non-blocking trigger to `POST /api/jobs/build_gold/start` on the monitoring platform.

Method	Endpoint	Emits	Description
POST	`/api/v1/stock/received`	`STOCK_RECEIVED`	Record goods receipt into a warehouse lot
POST	`/api/v1/stock/sold`	`STOCK_SOLD`	Record outbound sale; validates sufficient stock
POST	`/api/v1/stock/adjusted`	`STOCK_ADJUSTED`	Manual stock correction
POST	`/api/v1/stock/expired`	`STOCK_EXPIRED`	Write off an expired batch
GET	`/api/v1/stock/movements`	—	Recent movement history (default limit 50)

Idempotency

All stock endpoints accept an optional `Idempotency-Key: <uuid>` header. A repeated request carrying the same key within 24 hours returns the cached response without re-executing the operation or re-emitting to Kafka. This protects against duplicate events from client retries after network timeouts.

Reference data

`GET /api/v1/{products|suppliers|warehouses|inventory|inventory/risk}`

Reorder automation

Method	Endpoint	Description
GET	`/api/v1/reorder-policies`	List all active reorder policies
POST	`/api/v1/reorder-policies`	Create a policy (reorder_point, reorder_quantity, lead_time_days)
PUT	`/api/v1/reorder-policies/{id}`	Update thresholds
DELETE	`/api/v1/reorder-policies/{id}`	Deactivate (sets `active = FALSE`)

After each Gold pipeline run, `check_reorder_thresholds()` compares `fact_inventory_balance` against active `reorder_policies` and inserts `DRAFT` purchase orders for any product below its threshold.

Purchase order lifecycle

`DRAFT → APPROVED → SENT → RECEIVED / CANCELLED`

Method	Endpoint	Action
GET	`/api/v1/orders`	List purchase orders with optional status filter
POST	`/api/v1/orders/{po_id}/approve`	Approve a DRAFT order
POST	`/api/v1/orders/{po_id}/send`	Mark as SENT and dispatch supplier email (if SMTP configured)
POST	`/api/v1/orders/{po_id}/cancel`	Cancel a DRAFT or APPROVED order
POST	`/api/v1/orders/{po_id}/receive`	Mark as RECEIVED and emit a `STOCK_RECEIVED` Kafka event

Full request and response schemas: docs/api-reference.md

Monitoring Platform

Entry point: `python -m medwarehouse platform serve` (default port 8787)

Page	Path	Purpose
Dashboard	`/`	Aggregated health across all subsystems; active alert summary
Pipeline	`/pipeline`	Bronze → Warehouse freshness metrics and consistency gaps
Warehouse	`/warehouse`	DB reachability, table/view presence, row counts
Artifacts	`/artifacts`	Bronze, Silver, and Quarantine filesystem statistics
Infrastructure	`/infrastructure`	Docker service health; Start/Stop infrastructure buttons
Jobs	`/jobs`	Trigger and monitor predefined pipeline commands
Runbook	`/runbook`	In-app operator guide for diagnosing and recovering failures

The REST API at `/api/status` (and sub-paths `/pipeline`, `/infra`, `/alerts`) mirrors page data as JSON. Append `?refresh=hard` to bypass the 60-second TTL cache.

Available `job_id` values for `POST /api/jobs/{job_id}/start`:

```
warehouse_bootstrap  inventory_producer  inventory_bronze  inventory_silver
inventory_stage  refresh_dimensions  load_inventory_facts  refresh_views
quality_checks  build_gold  refresh_balance  check_reorders
procurement_producer  procurement_bronze  procurement_silver  build_procurement_gold
sales_producer  sales_bronze  sales_silver  build_sales_gold
```

Warehouse Schema

Dimensions (SCD Type 2)

All dimension tables carry `valid_from`, `valid_to`, `is_current`. Source data flows in from `master.*` via FDW. The `v_dim_*_current` views filter `WHERE is_current = TRUE`.

Table	Natural key	Tracked attributes
`analytics.dim_product`	`product_id`	`supplier_id`, `brand_name`, `generic_name`, `form_factor`, `hsn_code`
`analytics.dim_supplier`	`supplier_id`	`supplier_name`, `gstin`
`analytics.dim_warehouse`	`warehouse_id`	`warehouse_name`, `temperature_range`
`analytics.dim_customer`	`customer_id`	`customer_type`

Fact tables (range-partitioned by month)

Table	Domain	Contents
`analytics.fact_inventory_events`	Inventory	Append-only; one row per stock event
`analytics.fact_procurement_events`	Procurement	Append-only; one row per PO lifecycle event
`analytics.fact_sales_events`	Sales	Append-only; includes computed `line_revenue`

Operational tables

Table	Description
`analytics.fact_inventory_balance`	Physical inventory snapshot per product+warehouse+batch; refreshed after each Gold run
`analytics.reorder_policies`	Operator-configured reorder thresholds (`reorder_point`, `reorder_quantity`, `lead_time_days`)
`analytics.pending_purchase_orders`	Auto-generated DRAFT POs awaiting operator approval and dispatch

Key BI views

View	Description
`v_inventory_snapshot`	Non-zero stock balances enriched with product and warehouse names
`v_reorder_risk`	Products at or approaching their reorder threshold with risk status
`v_purchase_order_status`	Purchase orders with product and supplier names
`v_po_lifecycle`	PO progression with full lifecycle timestamps and computed lead time
`v_supplier_performance`	Fulfillment rate, fill rate, and average lead time per supplier
`v_revenue_summary`	Daily net revenue by product, warehouse, and customer type
`v_controlled_substance_register`	Dispensing register for Schedule H1/X/NDPS drugs (regulatory requirement)
`v_supplier_license_expiry`	License status per supplier: VALID / WARNING / CRITICAL / EXPIRED
`v_audit_activity`	Audit log enriched with username and role; `password_hash` excluded

Full schema reference: docs/data-dictionary.md

Security Model

Database roles

Role	Permissions
`spark_writer`	SELECT/INSERT/TRUNCATE on staging; INSERT on facts; EXECUTE on all warehouse procedures
`analytics_reader`	SELECT on all analytics tables and views
`fdw_reader`	SELECT on `medwarehouse_master` tables via FDW only

All warehouse procedures use `SECURITY DEFINER` — callers need only `EXECUTE` privilege, not direct table access.

API authentication

Both services authenticate via static API key. Keys are compared using `hmac.compare_digest` to prevent timing-based extraction.

```
X-API-Key: <key>
# or
Authorization: Bearer <key>
```

Set `MW_WEBAPP_API_KEY` and `MW_PLATFORM_API_KEY` in `.env`. Leaving either variable empty disables authentication for that service (acceptable in `local` development only).

Credential enforcement

`get_settings()` calls `_warn_weak_credentials()` at startup. In `local`/`dev` environments, weak default passwords (e.g. `postgres`) log a `WARNING`. In all other environments they raise `RuntimeError` immediately, preventing deployment with insecure defaults.

Full security model: docs/security.md

Airflow DAGs

Three DAGs with `schedule=None` (manual trigger only) and `max_active_runs=1`. Access the Airflow UI at **`http://localhost:8090`** (port remapped from the default 8080 to avoid conflict with the operator webapp).

`inventory_gold_pipeline` — 8 tasks

```
validate_silver_availability
        ↓
stage_inventory_events
        ↓
refresh_inventory_dimensions
        ↓
load_inventory_event_facts
        ↓
refresh_inventory_semantic_views
        ↓
inventory_quality_checks
        ↓
refresh_inventory_balance
        ↓
check_reorder_thresholds
```

`procurement_gold_pipeline` — 6 tasks

`validate_silver → stage → refresh_dims → load_facts → refresh_views → quality_checks`

`sales_gold_pipeline` — 6 tasks

`validate_silver → stage → refresh_dims → load_facts → refresh_views → quality_checks`

All DAGs share the same Python callables as the corresponding `orchestration build-*` CLI commands.

Tests

```
export PYTHONPATH=src
pytest tests/ -q
# 82 tests in ~0.6 s — no live infrastructure required
```

File	Coverage
`test_cli.py`	CLI argument parsing and dry-run producer
`test_config.py`	Settings construction and credential validation
`test_contracts.py`	Inventory event building and all validation rules
`test_domain_contracts.py`	Procurement and sales contract validation
`test_platform.py`	StatusService, ControlPlaneStore, ControlPlaneService, Flask routes
`test_sql_runner.py`	SQL template-variable rendering
`test_warehouse_functions.py`	Warehouse Python wrappers, auth middleware, credential warnings
`test_webapp_services.py`	Stock service (no `::uuid` casts), IdempotencyStore (TTL eviction + thread safety), TTLCache stampede protection, order service state machine, reorder SET clause construction

All tests run without live infrastructure. Integration tests that require a PostgreSQL connection are decorated `@pytest.mark.integration` and activated with `MW_INTEGRATION_TEST=1`.

Architecture

System Overview

MedWarehouse is an event-driven analytical warehouse for a medical/pharmaceutical operation. Business events produced by an operator webapp or sample producers flow through a three-stage Spark pipeline (Bronze → Silver → Gold) into a PostgreSQL analytical warehouse. Nine analytical domains are served to Power BI and to the operator webapp via a REST API.

Layer Map


Operator Webapp (port 8080)
Stock entry, reorder policies, purchase order management
writes to master DB + emits Kafka events
triggers pipeline via POST /api/jobs/build_*/start
medwarehouse_master (PostgreSQL OLTP)
products, suppliers, warehouse_locations, inventory_lots,
stock_adjustments, purchase_orders, goods_receipts, sales,
prescriptions, customers, users, payments, audit_logs
FDW (read-only) → master schema in analytics DB
Kafka Topics
inventory_events • procurement_events • sales_events
PySpark Structured Streaming
Bronze Layer (Parquet, immutable, append-only)
data/bronze/{inventory,procurement,sales}_events/
PySpark batch (validate, deduplicate, quarantine)
Silver Layer (Parquet, validated + deduped events)
data/silver/{inventory,procurement,sales}_events/
data/quarantine/{inventory,procurement,sales}_events/
PySpark JDBC write (truncate-and-load)
medwarehouse_analytics (PostgreSQL)
Staging: stg_inventory_events / stg_procurement_events /
stg_sales_events
Dimensions (SCD Type 2):
dim_product dim_supplier dim_warehouse dim_customer
Facts (partitioned by event_time):
fact_inventory_events fact_procurement_events
fact_sales_events
Operational: fact_inventory_balance reorder_policies
pending_purchase_orders
Semantic views (inventory, procurement, sales, compliance ...)
See data-dictionary.md for full view listing.
analytics_reader role

```
│  Power BI  /  Monitoring Platform (port 8787)                    │
└──────────────────────────────────────────────────────────────────┘
```

Kafka Domain Pipelines

Three complete Bronze → Silver → Gold pipelines, each independently orchestrated:

Domain	Topic	Event Types	Fact Table
Inventory	`inventory_events`	STOCK_RECEIVED, STOCK_SOLD, STOCK_ADJUSTED, STOCK_EXPIRED	`fact_inventory_events`
Procurement	`procurement_events`	PO_CREATED, PO_APPROVED, PO_RECEIVED, PO_CANCELLED	`fact_procurement_events`
Sales	`sales_events`	SALE_CREATED, SALE_CANCELLED	`fact_sales_events`

Each domain has: a Kafka schema, a Python contract, a Kafka producer, three Spark jobs (Bronze/Silver/Stage), a warehouse module, and an Airflow DAG.

Analytical Domains (FDW-backed)

Six additional domains read directly from the OLTP database via PostgreSQL FDW. No Spark pipeline.

Domain	Key Views
Prescriptions / Compliance	v_controlled_substance_register, v_prescription_compliance_violations
Supplier Management	v_supplier_license_expiry, v_supplier_directory
Financial / AP-AR	v_accounts_payable, v_daily_collections
Customer Analytics	v_customer_summary
Audit / Compliance	v_audit_activity, v_recalled_product_sales
Staff Performance	v_staff_performance, v_inactive_user_accounts

Security Model

Four PostgreSQL roles enforcing least-privilege. API key auth (`X-API-Key` header) on both Flask services. See `docs/security.md` for full details.

Package Structure

```
src/medwarehouse/
├── contracts/_utils.py          # Shared: deterministic_uuid, to_utc_iso
├── producers/common.py          # Shared: emit_events(), run_producer()
├── spark/jobs/_bronze.py        # Shared: run_bronze_ingestion()
│              _silver.py        # Shared: split_silver_quarantine()
│              _stage.py         # Shared: write_to_staging()
└── warehouse/_common.py         # Shared: validate_silver_ready(), call_warehouse_function()

src/medwarehouse_webapp/          # Flask operator webapp (port 8080)
```

Orchestration

Three Airflow DAGs (`schedule=None`). All can also be run via CLI or the monitoring platform job runner.

inventory_gold_pipeline (8 tasks): validate → stage → dims → facts → views → quality → balance → reorders

procurement_gold_pipeline (6 tasks): validate → stage → dims → facts → views → quality

sales_gold_pipeline (6 tasks): validate → stage → dims → facts → views → quality

---

Developer Guide

Table of Contents

1. System Purpose and Context
2. High-Level Architecture
3. Repository Layout
4. Configuration System
5. Data Flow: End-to-End
6. Layer 1 — Event Producers
7. Layer 2 — Apache Kafka
8. Layer 3 — Spark Bronze and Silver Pipelines
9. Layer 4 — PostgreSQL Analytical Warehouse
10. Layer 5 — Monitoring Platform and Operator Webapp
11. The Three Business Domains
12. Nine Analytical Domains in the Database
13. The Control Plane
14. Alert System
15. Airflow Orchestration
16. Infrastructure and Docker Compose
17. Testing Strategy
18. Key Design Decisions and Trade-offs
19. Important Nuances and Gotchas
20. Common Developer Workflows
21. Extending the System: Adding a New Domain

---

1. System Purpose and Context

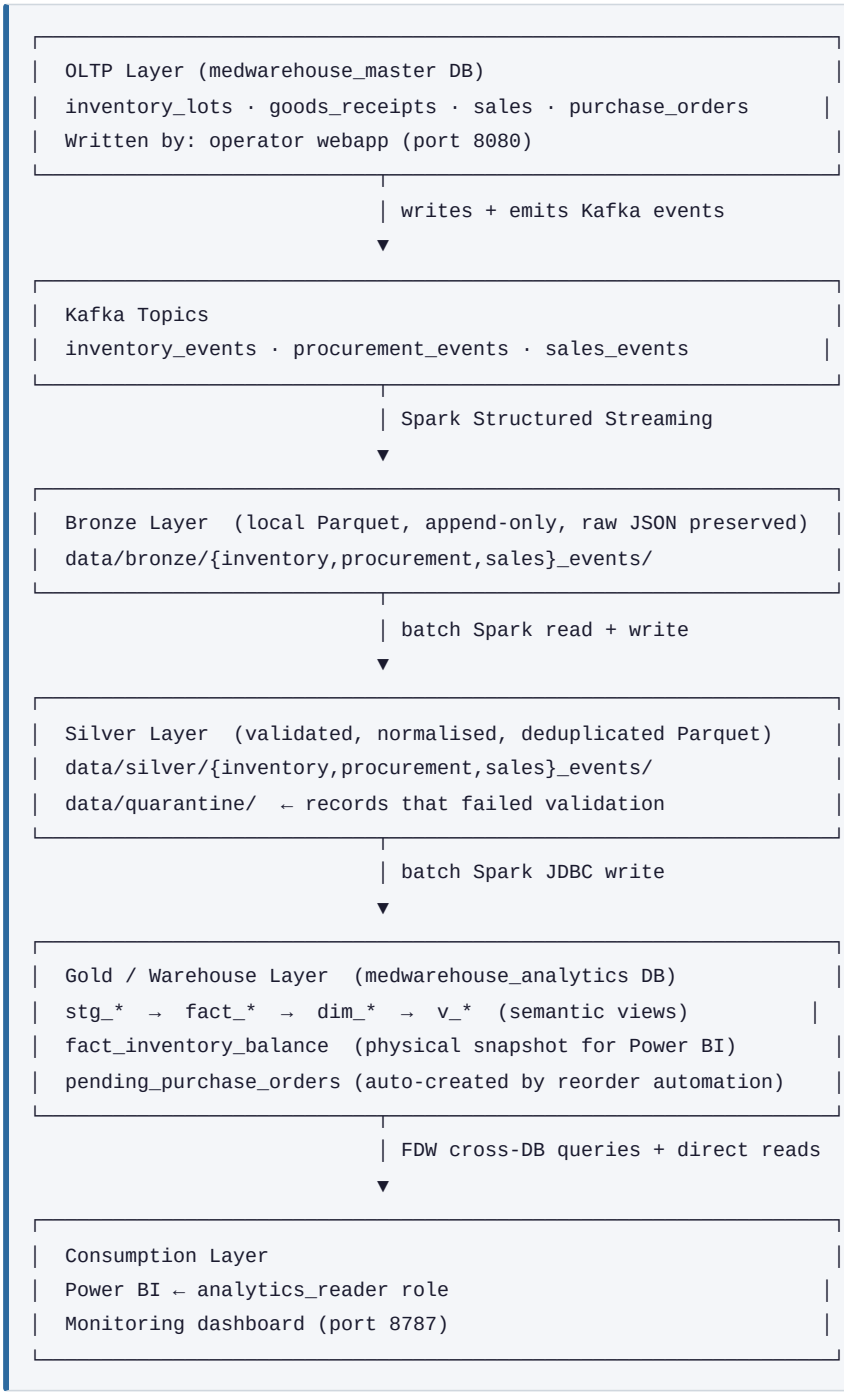
This repository implements a **medical/pharmaceutical inventory analytics warehouse**. A warehouse operator records stock movements (goods received, sold, adjusted, expired) via a REST API. Those events travel through a streaming pipeline and land in a read-optimised PostgreSQL schema where Power BI reports on inventory levels, reorder risk, procurement lifecycle, and sales velocity.

The system is event-sourced at its heart: inventory quantities are **never stored as a current balance**. Every stock movement is appended as an immutable event, and the current balance is derived by summing all events for a given product–warehouse–batch combination. This means you can reconstruct the state at any point in history by replaying events up to that timestamp.

The full stack involves six technologies working together:

Technology	Role
Python / Flask	Event producers, operator webapp REST API, monitoring dashboard
Apache Kafka	Durable event bus — decouples producers from consumers
Apache Spark (PySpark)	Streaming ingestion (Bronze), transformation (Silver), batch load (Gold)
PostgreSQL	Analytical warehouse with FDW, SCD-2 dimensions, partitioned facts
Apache Airflow	DAG-based orchestration of the Gold pipeline on demand
Docker Compose	Local development infrastructure

2. High-Level Architecture



The two Flask applications are independent processes:

- **Platform / Monitoring dashboard** (`python -m medwarehouse platform serve`, port 8787) — read-only operational view of the pipeline. Used by engineers.
- **Operator webapp** (`python -m medwarehouse_webapp`, port 8080) — data entry and purchase order management. Used by warehouse staff.

---

3. Repository Layout

```
.
├── airflow/dags/          # Airflow DAG definitions (one per domain)
├── data/                  # Runtime data (gitignored except .gitignore markers)
│   ├── bronze/            # Raw Parquet from Kafka
│   ├── silver/            # Validated Parquet
│   ├── quarantine/        # Failed-validation records
│   ├── gold/              # Legacy: pre-FDW gold outputs (no longer primary path)
│   └── control_plane/     # SQLite database for job run history
├── docker/
│   ├── airflow/Dockerfile # Airflow image with PySpark + project deps
│   └── postgres/init/     # DB creation + master data restore scripts
├── docs/                  # All project documentation
├── jars/                  # PostgreSQL JDBC driver for PySpark
├── schemas/kafka/         # JSON Schema for each Kafka event type
├── scripts/               # Shell scripts for local development
│   └── lib/common.sh      # Shared bash library sourced by all scripts
├── sql/warehouse/         # Ordered SQL files for warehouse bootstrap
├── src/
│   ├── medwarehouse/      # Core package: pipeline, platform, CLI
│   │   ├── cli.py         # Entry point: python -m medwarehouse <command>
│   │   ├── config.py      # All configuration via environment variables
│   │   ├── contracts/     # Event schemas and validation logic
│   │   ├── platform/      # Monitoring dashboard (Flask app + probes + alerts)
│   │   ├── producers/     # Kafka event generators (sample + live data)
│   │   ├── spark/         # PySpark Bronze/Silver/Stage jobs
│   │   └── warehouse/     # Python wrappers around PostgreSQL stored procedures
│   └── medwarehouse_webapp/  # Operator data-entry webapp (separate Flask app)
│       ├── api/           # REST API blueprints
│       └── services/      # Business logic (stock, orders, reorder policies)
└── tests/                 # Unit tests (no live DB or Kafka required)
```

---

4. Configuration System

File: `src/medwarehouse/config.py`

All configuration is read from environment variables at startup and assembled into a frozen `AppSettings` dataclass. There is no YAML, no INI file — everything goes through env vars.

Why environment variables?

The system targets Docker Compose locally and can trivially be adapted for Kubernetes or any container orchestration. Environment variables are the standard 12-factor config mechanism. Hardcoded config files require file mounts, templating, and update ceremonies; env vars just work.

The `get_settings()` singleton

```python
@lru_cache(maxsize=1)
def get_settings() -> AppSettings:
    ...
```

`get_settings()` is called once per process. `lru_cache(maxsize=1)` makes it a singleton. **This has an important implication for tests**: you must call `get_settings.cache_clear()` in `setUp`/`tearDown` when patching `os.environ`, otherwise the cached settings from a previous test bleed through.

Frozen dataclasses

`AppSettings` and all its sub-settings are `@dataclass(frozen=True)`. This prevents accidental mutation after construction. Immutability is especially important for settings shared across threads (the monitoring platform is multi-threaded).

Credential validation

`_warn_weak_credentials()` is called at the end of `get_settings()`. In `local`/`dev` environments, weak passwords like `"postgres"` log a `WARNING`. In any other environment they raise `RuntimeError` immediately — this prevents accidentally deploying with default credentials.

Key environment variables

Variable	Purpose	Default
`MW_ENV`	Environment name (affects credential validation)	`local`
`MW_MASTER_DB_*`	OLTP database (medwarehouse_master)	localhost:5432
`MW_ANALYTICS_ADMIN_DB_*`	Analytics DB (admin role)	localhost:5432
`MW_ANALYTICS_WRITER_DB_*`	Analytics DB (spark_writer role)	same host
`MW_ANALYTICS_READER_DB_*`	Analytics DB (analytics_reader role)	same host
`MW_KAFKA_BOOTSTRAP_SERVERS`	Kafka broker address	localhost:9092
`MW_PLATFORM_PORT`	Platform service port (used by webapp for pipeline trigger)	8787
`MW_WEBAPP_API_KEY`	API key for operator webapp	blank = no auth
`MW_PLATFORM_API_KEY`	API key for monitoring platform	blank = no auth
`MW_PLATFORM_SECRET_KEY`	Flask session secret for flash messages	random per-process

---

5. Data Flow: End-to-End

A full inventory stock receipt follows this path:

1. **Operator posts** `POST /api/v1/stock/received` to the webapp (port 8080).
2. Webapp writes a row to `inventory_lots` in the OLTP database (`medwarehouse_master`).
3. Webapp builds an `InventoryEvent` dict using `contracts.inventory.build_inventory_event()` and publishes it to the `inventory_events` Kafka topic.
4. **Bronze Spark job** (`spark inventory-bronze`) reads from Kafka as a stream and writes each raw JSON payload to Parquet in `data/bronze/inventory_events/`.
5. **Silver Spark job** (`spark inventory-silver`) reads Bronze Parquet, parses JSON, validates fields, deduplicates on `event_id`, writes clean events to `data/silver/inventory_events/` and bad records to `data/quarantine/inventory_events/`.
6. **Stage job** (`spark stage-inventory-events`) reads Silver Parquet and bulk-inserts into `analytics.stg_inventory_events` via JDBC.
7. **Warehouse Gold jobs** (called by Airflow DAG or directly):
   - `refresh_inventory_dimensions()` — upserts SCD-2 dimension rows.
   - `load_inventory_event_facts()` — moves staging events into `fact_inventory_events`, de-duplicating on `event_id`.
   - `refresh_inventory_semantic_views()` — refreshes materialised views.
   - `run_inventory_quality_checks()` — asserts referential integrity.
   - `refresh_inventory_balance()` — MERGEs current balances into `fact_inventory_balance`.

- `check_reorder_thresholds()` — inserts draft `pending_purchase_orders` for depleted stock.

8. **Power BI** queries `analytics.v_inventory_snapshot`, `analytics.fact_inventory_balance`, and `analytics.v_reorder_risk` via the `analytics_reader` role.

---

6. Layer 1 — Event Producers

Location: `src/medwarehouse/producers/`

There are two distinct producer paths:

Sample data producers

`inventory.py`, `procurement.py`, `sales.py` — generate deterministic synthetic events for development and testing. They are seeded from `SampleDataSettings` in config, so the same run always produces the same event IDs (important for deduplication tests). Run via:

```
python -m medwarehouse produce inventory --max-events 20
```

Live webapp producer

The operator webapp's `stock_service.py` calls `producers.common.emit_events()` directly after validating and writing each stock movement to the OLTP DB. This is the production code path.

`emit_events()` in `producers/common.py`

All producers funnel through `emit_events()`. It accepts an iterable of event dicts and handles both real Kafka publishing and `dry_run` mode (which logs instead of producing). The `confluent_kafka` import is lazy (inside the function body), which means the module is importable in test environments without Kafka installed.

---

7. Layer 2 — Apache Kafka

Topics: `inventory_events`, `procurement_events`, `sales_events`

Kafka is used as a durable, ordered, replay-capable event bus. The core reason for Kafka rather than writing directly to the DB is **decoupling**: the OLTP write (which must be fast and synchronous) is separated from the analytics pipeline (which can lag, catch up, and be re-run without loss).

Event schema

Each event follows this structure:

```
{
  "event_id": "uuid",
  "event_type": "STOCK_RECEIVED",
  "event_time": "2026-01-15T08:30:00+00:00",
  "producer": "webapp",
  "schema_version": 1,
  "payload": {
    "product_id": "...",
    "warehouse_id": "...",
    "batch_number": "...",
    "quantity_delta": 100,
    ...
  }
}
```

The `event_id` is deterministic (seeded UUID-v5) for sample data, and a timestamp-seeded UUID-v5 for webapp-originated events. This means replaying the same operation twice produces the same `event_id`, which allows the warehouse to deduplicate cleanly on the `ON CONFLICT DO NOTHING` clause in `load_inventory_event_facts()`.

Schema files

`schemas/kafka/{topic}.json` — JSON Schema definitions used for documentation and future schema registry integration.

8. Layer 3 — Spark Bronze and Silver Pipelines

Location: `src/medwarehouse/spark/`

Shared utilities

Three base modules (`_bronze.py`, `_silver.py`, `_stage.py`) contain the shared logic. Each domain's job (e.g., `inventory_bronze.py`) is a thin wrapper that passes domain-specific parameters.

Bronze — `_bronze.py`

Reads from Kafka using Spark Structured Streaming with `readStream`. Writes each Kafka message as a raw row with: - `raw_event` — the original JSON string (preserved for forensics) - `kafka_key`, `source_topic`, `source_partition`, `source_offset`, `source_kafka_timestamp` — provenance metadata

Why preserve raw JSON? If the Silver schema changes or a bug is found, you can re-derive Silver from Bronze without re-reading Kafka.

Bronze jobs run as **long-running streaming processes**. They are started in the background by the flow scripts and must be stopped after producers have flushed. The scripts use a `wait_for_parquet` function with a 90-second timeout to detect when Bronze has written output, then send SIGTERM.

Silver — `_silver.py`

Reads Bronze Parquet in batch mode. For each domain: 1. Parses the `raw_event` JSON column using the domain's event schema. 2. Adds `validation_errors` — an array of string codes for each failing rule. 3. Splits into `silver` (zero errors) and `quarantine` (one or more errors).

The quarantine split happens in `split_silver_quarantine()`:

```
silver     = df.filter(F.size("validation_errors") == 0)
quarantine = df.filter(F.size("validation_errors") > 0)
```

Silver records are also deduplicated on `dedupe_key` (= `event_id` if present, else SHA-256 of raw event).

Why write quarantine? Validation errors in streaming pipelines are silent by default. Writing failed records to a named quarantine path makes them inspectable and recoverable — an operator can fix the upstream issue and re-process quarantined records.

Stage — `_stage.py`

Reads Silver Parquet and uses Spark's JDBC writer to insert into `analytics.stg_*_events`. The staging table acts as a buffer — it is truncated before each load (via `TRUNCATE` in the stored procedure), so the fact load is always idempotent.

Why staging? Direct Silver-to-fact inserts with JDBC would require the Spark executor to resolve foreign keys (product_sk, warehouse_sk) at row level. Instead, the staging insert is a bulk append, and the `load_*_event_facts()` stored procedure joins staging against dimensions in PostgreSQL — much faster, and keeps complex join logic in SQL where it belongs.

9. Layer 4 — PostgreSQL Analytical Warehouse

Database: `medwarehouse_analytics`

The analytics schema is built by running the SQL files in `sql/warehouse/` in numeric order. `bootstrap_warehouse()` does this.

SQL file order

File	Contents
`01_roles_and_schemas.sql`	Creates roles (`fdw_reader`, `spark_writer`, `analytics_reader`), schemas (`master`, `analytics`)
`02_fdw_master_access.sql`	Creates the `postgres_fdw` extension, foreign server, user mappings, and imports the master DB tables as foreign tables under the `master` schema
`03_inventory_dimensions.sql`	SCD-2 dimension tables: `dim_product`, `dim_supplier`, `dim_warehouse`, `dim_customer`
`04_inventory_fact_and_staging.sql`	Staging table + range-partitioned `fact_inventory_events`
`05_inventory_loads_and_views.sql`	Stored procedures (`load_inventory_event_facts`, `refresh_inventory_dimensions`, etc.) and semantic views
`06_reorder_and_balance.sql`	`fact_inventory_balance`, `reorder_policies`, `pending_purchase_orders` tables
`07_reorder_functions.sql`	`refresh_inventory_balance()`, `check_reorder_thresholds()` stored procedures
`08_procurement_warehouse.sql`	Procurement staging, facts, views
`09_sales_warehouse.sql`	Sales staging, facts, views
`10_analytical_domains.sql`	Cross-domain views: supplier license expiry, controlled substance register, revenue summary

SCD Type-2 dimensions

Dimension tables like `dim_product` use Slowly Changing Dimension Type-2: each change creates a new row with a new `*_sk` surrogate key, with `valid_from` / `valid_to` / `is_current` flags. The `v_dim_*_current` views filter `WHERE is_current = TRUE` for convenience.

Why SCD-2? Point-in-time correctness. If a product's name changed on 1 March, a January sale must report with the January name. The fact table stores the surrogate key at the time of loading, so historical queries are automatically correct.

Foreign Data Wrapper (FDW)

`postgres_fdw` maps selected tables from `medwarehouse_master` into the `master` schema of `medwarehouse_analytics`. This means the analytics DB can JOIN its fact tables against live OLTP reference data without ETL duplication.

Trade-off: FDW queries incur a network round-trip to the master DB. For large analytical queries this is a bottleneck. The current design accepts this because the FDW tables are used primarily by stored procedures (not by direct Power BI queries), and the physical dimension tables cache the critical attributes.

Range-partitioned fact tables

`fact_inventory_events` is partitioned by `event_time` with monthly partitions. `ensure_inventory_fact_partitions()` is called before each load to create missing partitions automatically.

Why partitioning? Power BI date-range filters get partition pruning — queries for "last 30 days" skip all older partitions without a full scan. This is important at scale.

Roles and access control

Three application roles with least-privilege grants:

Role	Can do
`fdw_reader`	SELECT on master DB tables via FDW only
`spark_writer`	INSERT on staging, SELECT on dimensions, EXECUTE stored procedures
`analytics_reader`	SELECT on all analytics tables and views

Power BI connects as `analytics_reader`. Spark jobs connect as `spark_writer`. Stored procedures use `SECURITY DEFINER` so they run with the definer's privileges (postgres superuser), not the caller's.

---

10. Layer 5 — Monitoring Platform and Operator Webapp

Monitoring platform (`medwarehouse` package, port 8787)

Entry point: `python -m medwarehouse platform serve`

The platform is a Flask application structured around **probes** — classes that each collect a specific slice of system state.

Probe	What it collects
`InfraProbe`	Docker Compose service status via `docker compose ps`
`WarehouseProbe`	PostgreSQL reachability, table/view presence, row counts
`PipelineProbe`	Parquet file counts and timestamps for all Bronze/Silver/Quarantine paths, plus warehouse counts
`ArtifactProbe`	Per-domain filesystem artifact inventory
`JobProbe`	Run history from the SQLite control plane store
`AirflowProbe`	DAG existence and last-run state for all three DAGs

All six probes are collected in **parallel** via `ThreadPoolExecutor` in `StatusService._collect_probes()`. `WarehouseProbe` and `PipelineProbe` both need the same warehouse DB query; `StatusService` pre-fetches this once and shares the result with both to avoid a double roundtrip.

Results are assembled into a full status dict, then passed through the alert engine. The full status is cached with a 60-second TTL. The `TTLCache` implementation includes stampede protection via a per-key `threading.Event` — the first thread to encounter a cache miss claims the build slot; subsequent concurrent threads wait on the event rather than each launching their own expensive build.

Operator webapp (`medwarehouse_webapp` package, port 8080)

Entry point: `python -m medwarehouse_webapp`

REST API only (no HTML frontend — that is a separate project). The four stock endpoints each: 1. Validate the request body against `contracts.inventory` rules. 2. Write to the OLTP `inventory_lots` table in the master DB. 3. Emit a Kafka event via `producers.common.emit_events()`. 4. Fire a non-blocking async trigger to `POST /api/jobs/build_gold/start` on the platform service so the Gold pipeline runs.

Idempotency: Every POST endpoint supports an `Idempotency-Key` header. If the same key is seen twice within 24 hours, the cached response is returned without re-executing. This protects against client retries after timeouts. The `IdempotencyStore` is an in-memory dict with a background eviction thread. In production, replace it with Redis.

Reorder automation: After Gold pipeline execution, `check_reorder_thresholds()` compares `fact_inventory_balance` against `reorder_policies` and inserts `DRAFT` purchase orders for any product below its threshold. Operators review draft POs at `GET /api/v1/orders`, then approve → send → receive. The `send` step dispatches a supplier email via SMTP if `MW_SMTP_HOST` is configured. The `receive` step emits a new `STOCK_RECEIVED` event, closing the loop.

---

11. The Three Business Domains

Each domain maps to a set of Kafka events, Spark jobs, warehouse tables, and an Airflow DAG.

Inventory

The primary domain. Tracks stock movements in a warehouse: - `STOCK_RECEIVED` — goods arrive, quantity goes up - `STOCK_SOLD` — goods dispatched to a customer, quantity goes down - `STOCK_ADJUSTED` — manual correction (damaged goods, audit discrepancy) - `STOCK_EXPIRED` — batch expired, written off

Current balance = sum of all `quantity_delta` values for a product–warehouse–batch.

Procurement

Tracks the purchase order lifecycle from a supplier's perspective: - `PO_CREATED` — a purchase order is raised - `PO_APPROVED` — internally authorised - `PO_SENT` — transmitted to supplier - `PO_RECEIVED` — goods physically received (triggers an inventory receipt) - `PO_CANCELLED` — order withdrawn

Sales

Tracks outbound customer sales: - `SALE_CREATED` — sale confirmed - `SALE_CANCELLED` — reversed - `SALE_RETURNED` — customer return

---

12. Nine Analytical Domains in the Database

The AI-generated master database (`meddata_full_20250916_175921.dump`) contains nine business domains covering a realistic pharmaceutical operation. These are exposed through the FDW and semantic views:

Domain	Key Tables / Views	Business Question Answered
Inventory	`fact_inventory_events`, `v_inventory_snapshot`, `fact_inventory_balance`	What stock do we hold right now, and how has it moved?
Procurement	`fact_procurement_events`, `v_po_lifecycle`, `v_supplier_performance`	Which POs are open, and how do suppliers perform on lead time?
Sales	`fact_sales_events`, `v_revenue_summary`	What is our revenue by product and time period?
Compliance	`v_controlled_substance_register`, `v_supplier_license_expiry`	Are controlled substances properly tracked? Are supplier licences current?
Supplier	`dim_supplier`, `v_dim_supplier_current`	Supplier master data with SCD-2 history
Product	`dim_product`, `v_dim_product_current`	Product catalogue with pricing and classification
Warehouse	`dim_warehouse`, `v_dim_warehouse_current`	Warehouse locations and temperature capabilities
Customer	`dim_customer`	Dispensary and hospital customers
Financial	`v_revenue_summary`, reorder cost estimates in `v_reorder_risk`	Revenue vs. cost of goods

---

13. The Control Plane

Location: `src/medwarehouse/platform/control/`

The control plane is the system that allows the monitoring dashboard to start, stop, and track pipeline jobs. It has three components:

`ControlPlaneStore` (SQLite)

SQLite in WAL mode is used because the control plane is a local tool running on a developer's machine — there is no need for a full PostgreSQL deployment just for job tracking. WAL mode allows concurrent readers while a single writer commits.

Per-thread connections via `threading.local()` eliminate the overhead of opening a connection per method call. The schema is two tables: `runs` (one row per job execution) and `run_logs` (one row per log line, foreign-keyed to `runs` with `ON DELETE CASCADE`). Runs older than 30 days are pruned at startup to prevent unbounded growth.

`ProcessSupervisor`

Launches pipeline commands as subprocesses with `start_new_session=True`, which puts each job in its own process group. This allows sending SIGTERM/SIGKILL to the entire group (preventing orphaned child processes if Spark spawns workers).

Stop sequence: SIGTERM → wait up to 10 seconds → SIGKILL. A single watcher thread handles both natural exit detection and SIGKILL escalation (earlier designs used two separate threads, creating a race condition on the final store update — merged into one to eliminate the race).

On startup, `_reconcile_runs()` is called to handle the case where the control plane was restarted while a job was running. Active runs without a live PID are marked `failed`; active runs with a live PID are marked `orphaned` (so operators can decide what to do).

`ControlPlaneService`

Public API used by Flask routes: `start_job(job_id)`, `stop_job(job_id)`, `run_infra_action("start"|"stop")`. All operations are protected by a `threading.Lock` to prevent double-starts from concurrent HTTP requests.

`JobSpec` catalog

`catalog.py` contains the complete list of predefined jobs as frozen `JobSpec` dataclasses. A module-level dict provides O(1) lookup. The `command` field is the argv suffix after `python -m medwarehouse`, so jobs are just CLI commands — no magic.

---

14. Alert System

Location: `src/medwarehouse/platform/services/alerts/`

The alert system is a rules engine: a list of `AlertRule` instances that each receive the full metrics dict and return an `Alert` or `None`.

Split into submodules

The rules are split by concern:

Module	Rules
`inventory.py`	Freshness, volume anomaly, quarantine volume, pipeline consistency
`infra.py`	Warehouse reachability, Kafka/PostgreSQL service status
`airflow.py`	DAG failure, DAG staleness
`compliance.py`	Supplier licence expiry, controlled substance compliance

`AlertEngine`

Manages state transitions: an alert transitions from absent → `ACTIVE` → `RESOLVED` as conditions appear and clear. Recent resolved alerts are kept in a deque for history. The engine evaluates rules against the current metrics snapshot on every status refresh.

`VolumeBaselineStore`

`VolumeAnomalyRule` detects unusual drops in pipeline volume by comparing the current count against a rolling average of the last N observations (default 6). The baseline is accumulated in `VolumeBaselineStore`, which lives in `_engine.py` as the canonical definition. `inventory.py` imports it from there — there is only one class, not two.

---

15. Airflow Orchestration

Location: `airflow/dags/`

Three DAGs, one per domain. All DAGs have `schedule=None` (manual trigger only) and `max_active_runs=1` (prevents concurrent runs that would conflict on staging tables).

Inventory DAG task chain

```
validate_silver → stage_events → refresh_dimensions → load_facts
  → refresh_views → quality_checks → refresh_balance → check_reorders
```

The quality checks step calls `run_inventory_quality_checks()` which raises `RuntimeError` if any check fails — this causes the DAG task to fail and stops the chain, preventing bad data from propagating.

Why Airflow?

Airflow is used as a scheduler/orchestrator rather than running the Python functions directly. This provides: retry logic, task-level success/failure tracking, dependency enforcement between tasks, and the Airflow UI for historical run inspection.

Trade-off: Airflow adds significant infrastructure weight (a PostgreSQL metadata DB, webserver, scheduler, worker). For this local deployment the same pipeline can be run via the CLI (`python -m medwarehouse orchestration build-gold`) or from the monitoring dashboard. Airflow becomes important when running on a schedule or in a shared environment.

---

16. Infrastructure and Docker Compose

Service topology

Service	Port	Purpose
`postgres`	5432	Both `medwarehouse_master` and `medwarehouse_analytics` databases
`zookeeper`	2181	Kafka coordination (deprecated in newer Kafka but present in Confluent 7.5)
`kafka`	9092 (external), 29092 (internal)	Event bus
`airflow-webserver`	**8090**	Airflow UI (remapped from default 8080 to avoid conflict with webapp)
`airflow-scheduler`	—	Airflow scheduling backend

Port 8090 for Airflow: The standard Airflow port is 8080. The operator webapp also runs on 8080. They cannot both bind to the same host port, so Airflow is remapped to 8090 in this deployment.

Health checks and `depends_on` conditions

PostgreSQL and Kafka both have `healthcheck` blocks. The Airflow services use `condition: service_healthy` and `condition: service_completed_successfully` so they do not start until the dependencies are actually ready (not just started).

This is important because Kafka takes 15–30 seconds to elect a leader after the container starts. Without health checks, `airflow-init` would fail trying to connect to Kafka before it's ready.

Restart policies

Core services (postgres, kafka, zookeeper, and Airflow application services) have `restart: unless-stopped`. This means they survive host reboots and `docker daemon` restarts without manual intervention.

Database initialisation

`docker/postgres/init/` contains two SQL files and a shell script that run on first container startup (PostgreSQL's `docker-entrypoint-initdb.d` mechanism): 1. `00-create-databases.sql` — creates `medwarehouse_master`, `medwarehouse_analytics`, and `airflow` databases. 2. `01-create-airflow-user.sql` — creates the Airflow PostgreSQL user. 3. `02-restore-master.sh` — restores `meddata_full_20250916_175921.dump` into `medwarehouse_master`.

The master database restore is the AI-generated sample dataset. It is restored once at container creation and not re-run.

---

17. Testing Strategy

Philosophy

All tests in `tests/` run without a live database, Kafka cluster, or Spark session. Heavy dependencies (`psycopg2`, `confluent_kafka`, `pyspark`) are imported **lazily** — inside function bodies rather than at module level. This means `import medwarehouse.warehouse.inventory` succeeds in a minimal test environment without those packages installed, and tests can mock the DB layer cleanly.

Critical pattern: `__init__.py` files throughout the package are deliberately empty (no eager imports). This prevents a test that imports `medwarehouse.warehouse` from pulling in psycopg2 transitively.

Test files

File	What it tests
`test_cli.py`	CLI entry point smoke test (dry-run inventory producer)
`test_config.py`	Settings construction, weak credential detection, API key env vars
`test_contracts.py`	Inventory event building and validation rules
`test_domain_contracts.py`	Procurement and sales contract validation
`test_platform.py`	StatusService, ControlPlaneStore, ControlPlaneService, Flask route layer
`test_sql_runner.py`	SQL template rendering
`test_warehouse_functions.py`	Warehouse Python wrappers (mocking the DB), credential warnings, auth middleware
`test_webapp_services.py`	Stock service (writes + events), order service state machine, reorder service SET clause, IdempotencyStore, TTLCache stampede protection

Running tests

```
pytest tests/ -q               # all 82 unit tests
pytest tests/ -k "idempotency"  # subset by keyword
MW_INTEGRATION_TEST=1 pytest tests/test_warehouse_functions.py -m integration  # live DB
```

Mock targets

When mocking the DB layer, mock the `fetch_rows` or `connect` function at the point of use, not at the import site:

```
# Correct: mock at usage site
patch("medwarehouse.warehouse._common.fetch_rows")

# Wrong: won't intercept calls that imported the name already
patch("medwarehouse.warehouse.inventory.fetch_rows")
```

---

18. Key Design Decisions and Trade-offs

Event sourcing for inventory

Decision: Never store a `current_quantity` column that gets updated in place. Instead, derive it from the sum of event deltas.

Why: Append-only event logs are auditable, replayable, and correct under concurrent writes. A running balance stored as a column is prone to race conditions under concurrent updates (two SOLD events for the same lot running simultaneously would both read the same balance and both decrement from it, losing one update).

Trade-off: Querying current balance requires summing events, which is slower than a point lookup. This is mitigated by `fact_inventory_balance` — a physical snapshot maintained by `refresh_inventory_balance()` that Power BI reads instead of the derived view.

Dual-database architecture

Decision: OLTP data lives in `medwarehouse_master`; analytics live in `medwarehouse_analytics`. They communicate via FDW.

Why: Analytical queries (aggregations, window functions, cross-joins) compete with OLTP writes for resources. Separating them onto different schemas/connections means a slow Power BI query cannot lock an inventory_lots row that an operator needs to update.

Trade-off: FDW adds latency and a single point of coupling between the two databases. If the master DB is unreachable, the analytics stored procedures that query FDW tables will fail.

Local Parquet as the intermediate store

Decision: Bronze and Silver are local Parquet files, not database tables or Kafka streams.

Why: Parquet is self-describing, columnar, and cheap to re-read. Storing as files means the Silver transformation can be re-run any number of times against the same Bronze data without re-reading Kafka. This is important for recovery: if a Silver job has a bug, fix the code, delete the Silver directory, and re-run Silver from Bronze.

Trade-off: This architecture works only when Bronze and Silver jobs run on the same machine. In a distributed deployment, a shared filesystem (S3, HDFS) would replace local paths.

SQLite for the control plane

Decision: The control plane store (job run history) uses SQLite rather than PostgreSQL.

Why: The monitoring platform is a local developer tool. Adding a dependency on the analytics database would mean the monitoring platform cannot start if the analytics DB is down — defeating its purpose as a tool to diagnose infrastructure problems. SQLite is self-contained, requires no setup, and its WAL mode handles the modest multi-reader/single-writer concurrency this application needs.

Deterministic event IDs

Decision: Sample data event IDs are UUID-v5 seeded from a string (`seed + event_name`). Webapp event IDs are UUID-v5 seeded from `seed + operation + timestamp`.

Why: The warehouse uses `ON CONFLICT (event_id) DO NOTHING` to deduplicate. For this to work reliably on retries, the same logical operation must produce the same `event_id`. UUID-v5 is deterministic given the same inputs, so re-submitting a failed request (with an `Idempotency-Key`) will produce an event that the database safely ignores as a duplicate.

Separate monitoring and operator apps

Decision: Two completely independent Flask applications.

Why: Their security models differ. The monitoring platform shows sensitive operational data and should be accessible only to engineers; the operator webapp handles data entry and should be accessible to warehouse staff. Running them as separate processes means separate API keys, separate logs, and independent restart/scaling.

---

19. Important Nuances and Gotchas

Product and warehouse IDs are TEXT, not UUID

The OLTP schema uses TEXT columns for `product_id` and `location_id`. Sample values look like `"P-LOCAL-CALPOL-500"`. Do not add `::uuid` PostgreSQL casts in any SQL query — they will fail with `invalid input syntax for type uuid` for non-UUID strings.

`get_settings.cache_clear()` in tests

The `get_settings()` singleton caches on first call. Any test that patches `os.environ` must call `get_settings.cache_clear()` in both `setUp` and `tearDown`, otherwise patched environment variables bleed through into subsequent tests that assume clean settings.

Bronze lifecycle in scripts

The Bronze Spark job is a long-running streaming process. Flow scripts manage its lifecycle: 1. Start Bronze in the background (`&`). 2. Set a `trap ... EXIT` to kill it on script exit. 3. Produce events. 4. Wait for Parquet output with `wait_for_parquet` (polling with 90-second timeout). 5. Stop Bronze cleanly before running Silver.

If you abort a flow script mid-run, the EXIT trap handles cleanup. But if the script is killed with SIGKILL (`kill -9`), the trap does not run — you will need to manually kill the Bronze PID.

Airflow `--if-not-exists` on user create

The `airflow users create` command in `airflow-init` uses `--if-not-exists` (available in Airflow 2.8+, which is the minimum version). This prevents the init container from failing on `docker compose up` re-runs when the user already exists.

Parquet overwrite in Silver

Silver writes with `mode("overwrite")`. This means running Silver twice on the same Bronze data is idempotent — the second run replaces the first. This is intentional (Silver is derived from Bronze; it is not authoritative and can be regenerated at any time). The warehouse Gold pipeline reads Silver once; after that, Silver can be safely deleted to save disk space.

`SECURITY DEFINER` on stored procedures

Stored procedures in the analytics schema use `SECURITY DEFINER`, which means they execute with the privileges of their creator (postgres superuser) regardless of which role calls them. This allows `spark_writer` to call `load_inventory_event_facts()` even though `spark_writer` does not have direct INSERT privileges on `fact_inventory_events` (which is owned by postgres). Keep this in mind when debugging permission errors — the effective user inside a `SECURITY DEFINER` function is the definer, not the caller.

`ProducerSettings` vs `SampleDataSettings`

These were deliberately separated during a refactor. `ProducerSettings` has only three fields: `interval_seconds`, `max_events`, `dry_run` — runtime controls. `SampleDataSettings` has all the seed values used to generate repeatable test data. Do not add sample-data-specific fields to `ProducerSettings`; they belong in `SampleDataSettings`.

`VolumeBaselineStore` has one canonical location

`VolumeBaselineStore` is defined in `alerts/_engine.py`. `alerts/inventory.py` imports it from there. The class must not be re-defined in `inventory.py` — both modules share the same class so that `isinstance` checks and baseline stores passed between them are consistent.

Thread safety of the TTLCache

`TTLCache` uses a `threading.Event` per cache key to prevent concurrent builds (thundering herd). The design guarantees exactly one builder per concurrent miss. However, `invalidate()` does not wait for an in-progress build to finish — if a force-refresh races with a cache write, the freshly built value might be evicted immediately. This is acceptable for a monitoring dashboard (the next request will just rebuild), but it would be problematic for a high-write-rate cache.

---

20. Common Developer Workflows

First-time setup

```
cd Event-Driven-Inventory-Analytics-Warehouse-BI-System
cp .env.example .env     # review and adjust if needed
./scripts/setup_local_stack.sh
```

This script checks prerequisites, creates a venv, starts Docker services, and bootstraps the warehouse schema.

Run the inventory pipeline end-to-end

```
./scripts/run_inventory_flow.sh
```

This handles: Kafka topic creation → Bronze start → event production → wait for Parquet → Bronze stop → Silver → Stage → Gold (dimensions + facts + views + quality checks + balance refresh + reorder check).

Start the monitoring dashboard

```
source .venv/bin/activate
python -m medwarehouse platform serve --port 8787
```

Open `http://localhost:8787`.

Start the operator webapp

```
source .venv/bin/activate
python -m medwarehouse_webapp --port 8080
```

Run tests

```
source .venv/bin/activate
pip install -e ".[dev]"
pytest tests/ -q
```

Bootstrap or re-bootstrap the warehouse schema

```
python -m medwarehouse warehouse bootstrap
```

Runs all SQL files in `sql/warehouse/` in order. Safe to re-run (`CREATE OR REPLACE` and `IF NOT EXISTS` are used throughout).

Inspect quarantined records

```
python -c "
import pandas as pd
df = pd.read_parquet('data/quarantine/inventory_events/')
print(df[['event_id','event_type','validation_errors']].head(20))
"
```

Trigger the Gold pipeline manually (without Airflow)

```
python -m medwarehouse orchestration build-gold
```

---

21. Extending the System: Adding a New Domain

To add a new domain (e.g., `returns`):

1. **Define the event contract** — Add `src/medwarehouse/contracts/returns.py` following the same pattern as `inventory.py`: define event types, required fields, build function, and validate function.
2. **Add a Kafka topic** — Add `MW_KAFKA_RETURNS_TOPIC=returns_events` to `.env.example` and `config.py` (`KafkaSettings`).
3. **Create Spark jobs** — Add `returns_bronze.py`, `returns_silver.py`, `returns_stage.py` in `src/medwarehouse/spark/jobs/`, each delegating to the shared `_bronze.py`, `_silver.py`, `_stage.py` utilities with domain-specific schema and validation.
4. **Add path settings** — Add `bronze_returns_path`, `silver_returns_path`, etc. to `PathSettings` in `config.py`.
5. **Create a producer** — Add `src/medwarehouse/producers/returns.py` using `run_producer()` from `common.py`.
6. **Write SQL** — Add `11_returns_warehouse.sql` with staging table, fact table, stored procedures, and semantic views.
7. **Add warehouse Python wrappers** — Add `src/medwarehouse/warehouse/returns.py` with `validate_returns_silver_ready()`, `load_returns_event_facts()`, etc.

8. **Register CLI commands** — In `cli.py`, add entries to the `spark`, `warehouse`, and `orchestration` subparsers, and update `_warehouse_dispatch`.
9. **Create an Airflow DAG** — Add `airflow/dags/returns_gold_pipeline.py` following the same task chain as the inventory DAG.
10. **Register jobs in the catalog** — Add `JobSpec` entries to `platform/control/catalog.py`.
11. **Update probes** — Add the returns paths to `ArtifactProbe` and `PipelineProbe`.
12. **Add freshness alert rules** — Register a `FreshnessAlertRule` for returns bronze and silver in `build_default_alert_engine()`.
13. **Add flow script** — Create `scripts/run_returns_flow.sh` sourcing `lib/common.sh`.
14. **Write tests** — Cover the contract validation, the warehouse wrapper, and at minimum a smoke test for the producer.

The pattern is deliberately consistent across domains so that adding a new one is mechanical, not creative.

Deployment Guide

This guide covers deploying the full medwarehouse stack locally and to a shared server.

Prerequisites

Requirement	Minimum Version	Notes
Docker	24.x	Required for all infrastructure services
Docker Compose	2.x	Included with Docker Desktop
Python	3.11	3.12 and 3.13 supported
Java JRE	17	Required by PySpark. Install `openjdk-17-jre-headless`
Git	2.x	

Java installation (Ubuntu/Debian):

```
sudo apt-get install -y openjdk-17-jre-headless
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Environment Configuration

Copy `.env.example` to `.env` and configure all values:

```
cp .env.example .env
```

Mandatory settings to change before any shared deployment:

Variable	Description	How to generate
`MW_MASTER_DB_PASSWORD`	Master DB password	`openssl rand -hex 16`
`MW_ANALYTICS_ADMIN_DB_PASSWORD`	Analytics admin password	`openssl rand -hex 16`
`MW_ANALYTICS_WRITER_DB_PASSWORD`	Spark writer password	`openssl rand -hex 16`
`MW_ANALYTICS_READER_DB_PASSWORD`	BI reader password	`openssl rand -hex 16`
`MW_ANALYTICS_FDW_READER_PASSWORD`	FDW password	`openssl rand -hex 16`
`MW_WEBAPP_API_KEY`	Webapp auth key	`openssl rand -hex 32`
`MW_PLATFORM_API_KEY`	Monitoring platform auth key	`openssl rand -hex 32`

> **Security note:** The system raises a `RuntimeError` at startup if any password matches a known weak default in any environment except `local` or `dev`. Set `MW_ENV` to a value other than `local` / `dev` in all shared deployments.

Infrastructure Startup

Start core services (PostgreSQL + Kafka)

```
./scripts/start_services.sh
```

This script: 1. Starts PostgreSQL, Zookeeper, and Kafka via Docker Compose 2. Waits for Kafka to be ready 3. Creates the three Kafka topics: `inventory_events`, `procurement_events`, `sales_events`

Start with Airflow

```
./scripts/start_services.sh --with-airflow
```

Airflow initialises its database, creates an admin user (credentials set via `AIRFLOW_ADMIN_PASSWORD` in `.env`, defaulting to `admin`), and starts the webserver and scheduler. The Airflow webserver binds to host port **8090** (not the default 8080) to avoid conflict with the operator webapp on port 8080.

Stop all services

```
./scripts/stop_services.sh
```

Python Environment Setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt       # Full lockfile (for Airflow container parity)
# OR for local dev:
pip install -e ".[spark,dev]"         # Uses pyproject.toml
```

Set `PYTHONPATH` so the medwarehouse package is discoverable:

```
export PYTHONPATH="$PWD/src"
```

Or source the `.env` file (it does not set PYTHONPATH; the scripts set it automatically):

```
set -a && source .env && set +a
```

Warehouse Bootstrap

Run once after starting PostgreSQL to create all schemas, roles, FDW, dimensions, facts, views, and functions:

```
python -m medwarehouse warehouse bootstrap
```

This executes all SQL files in `sql/warehouse/` in lexicographic order (01 → 10).

> **Important:** The bootstrap SQL uses template substitution for role names and passwords. Ensure all `MW_*` environment variables are set before running bootstrap.

Running the Inventory Pipeline (Step-by-Step)

```
# 1. Produce sample inventory events to Kafka
python -m medwarehouse produce inventory --max-events 20

# 2. In a separate terminal — start the Bronze stream (runs continuously)
python -m medwarehouse spark inventory-bronze

# 3. After events land in Bronze, run Silver transformation
python -m medwarehouse spark inventory-silver

# 4. Build Gold (stage → load facts → refresh views → quality checks → balance → reorder check)
python -m medwarehouse orchestration build-gold
```

Or use the convenience script:

```
./scripts/run_inventory_flow.sh 20    # produces 20 events and runs the full pipeline
```

Running the Procurement Pipeline

```
python -m medwarehouse produce procurement --max-events 15
python -m medwarehouse spark procurement-bronze   # separate terminal, long-running
python -m medwarehouse spark procurement-silver
python -m medwarehouse orchestration build-procurement-gold
```

Running the Sales Pipeline

```
python -m medwarehouse produce sales --max-events 20
python -m medwarehouse spark sales-bronze          # separate terminal, long-running
python -m medwarehouse spark sales-silver
python -m medwarehouse orchestration build-sales-gold
```

Starting the Operator Webapp

```
python -m medwarehouse_webapp --host 0.0.0.0 --port 8080
```

If `MW_WEBAPP_API_KEY` is set in `.env`, all API calls require:

```
X-API-Key: <your key>
```

or

```
Authorization: Bearer <your key>
```

The `/health` endpoint is always accessible without authentication.

Starting the Monitoring Platform

```
python -m medwarehouse platform serve --host 0.0.0.0 --port 8787
```

Accessible at `http://localhost:8787`. All pages and API endpoints require `MW_PLATFORM_API_KEY` if set.

Running via Airflow

Three DAGs are available after Airflow starts:

DAG ID	Trigger	Description
`inventory_gold_pipeline`	Manual	Full inventory Gold pipeline (8 tasks, includes balance refresh + reorder check)
`procurement_gold_pipeline`	Manual	Full procurement Gold pipeline (6 tasks)
`sales_gold_pipeline`	Manual	Full sales Gold pipeline (6 tasks)

Access Airflow UI at **`http://localhost:8090`** (credentials: `admin` / value of `AIRFLOW_ADMIN_PASSWORD`, default `admin`).

Port Reference

Service	Port	Notes
PostgreSQL	5432	Hosts both `medwarehouse_master` and `medwarehouse_analytics`
Kafka	9092	External listener (localhost)
Kafka internal	29092	Docker network listener
ZooKeeper	2181	Kafka coordination
Airflow webserver	**8090**	DAG management UI (remapped from default 8080 — see note below)
Monitoring platform	8787	Operational monitoring dashboard
Operator webapp	8080	Data entry and PO management

> **Port note:** The Airflow webserver is mapped to host port **8090** in `docker-compose.yml` to avoid conflict with the operator webapp, which runs on port 8080. Do not change the operator webapp to 8090 unless you also change the Airflow mapping.

Environment Variables Reference

See `.env.example` for the complete annotated list. Key variables:

Variable	Default	Description
`MW_ENV`	`local`	Environment name. Non-local environments enforce strong credentials.
`MW_KAFKA_BOOTSTRAP_SERVERS`	`localhost:9092`	Kafka broker address
`MW_MASTER_DB_*`	localhost/5432/postgres	Master database connection
`MW_ANALYTICS_ADMIN_DB_*`	localhost/5432/postgres	Analytics admin connection
`MW_ANALYTICS_WRITER_DB_*`	—	Spark writer connection
`MW_ANALYTICS_READER_DB_*`	—	BI reader connection
`MW_WEBAPP_API_KEY`	(empty)	Webapp auth key. Empty = no auth (dev only).
`MW_PLATFORM_API_KEY`	(empty)	Platform auth key. Empty = no auth (dev only).
`MW_SMTP_HOST`	(empty)	SMTP server for supplier emails. Empty = log only.
`JAVA_HOME`	—	Required for PySpark. Must point to Java 17 installation.

Upgrading

1. Pull the latest code
2. Run `pip install -r requirements.txt` to pick up any new Python dependencies
3. Run `python -m medwarehouse warehouse bootstrap` to apply new SQL files
4. Restart all running services

Health Checks

```
# Platform health
curl http://localhost:8787/health

# Webapp health
curl http://localhost:8080/health

# Warehouse quality checks
python -m medwarehouse warehouse quality-checks
python -m medwarehouse warehouse procurement-quality-checks
python -m medwarehouse warehouse sales-quality-checks

# Pipeline status
curl http://localhost:8787/api/status/pipeline
```

Development Guide

Everything needed to set up a local development environment, run tests, and understand the codebase structure.

Repository Structure

```
medwarehouse/
├── src/
│   ├── medwarehouse/              # Main Python package
│   │   ├── config.py              # All settings loaded from environment
│   │   ├── logging.py             # Logging setup
│   │   ├── cli.py                 # CLI entrypoint: python -m medwarehouse <command>
│   │   ├── contracts/             # Event schemas and validation
│   │   │   ├── inventory.py       # STOCK_* event contract
│   │   │   ├── procurement.py     # PO_* event contract
│   │   │   └── sales.py           # SALE_* event contract
│   │   ├── producers/             # Kafka event producers
│   │   │   ├── common.py          # emit_events() + Kafka producer
│   │   │   ├── inventory.py
│   │   │   ├── procurement.py
│   │   │   └── sales.py
│   │   ├── spark/
│   │   │   ├── session.py         # build_spark_session()
│   │   │   └── jobs/
│   │   │       ├── inventory_bronze.py    # Kafka → Bronze Parquet
│   │   │       ├── inventory_silver.py    # Bronze → Silver Parquet
│   │   │       ├── inventory_stage.py     # Silver → stg_inventory_events
│   │   │       ├── procurement_bronze.py
│   │   │       ├── procurement_silver.py
│   │   │       ├── procurement_stage.py
│   │   │       ├── sales_bronze.py
│   │   │       ├── sales_silver.py
│   │   │       └── sales_stage.py
│   │   ├── warehouse/
│   │   │   ├── postgres.py        # psycopg2 connection context manager
│   │   │   ├── sql_runner.py      # execute_sql_file, fetch_rows, execute_statement
│   │   │   ├── inventory.py       # Inventory warehouse functions
│   │   │   ├── procurement.py     # Procurement warehouse functions
│   │   │   └── sales.py           # Sales warehouse functions
│   │   └── platform/              # Flask monitoring app (port 8787)
│   │       ├── api/               # Flask blueprints
│   │       ├── auth.py            # API key middleware
│   │       ├── control/           # Job runner (SQLite-backed)
│   │       ├── models/            # Dataclasses
│   │       ├── probes/            # Data collection probes
│   │       ├── services/          # StatusService, AlertEngine
│   │       └── ui/                # Jinja2 templates + static files
│   └── medwarehouse_webapp/       # Operator webapp (port 8080)
│       ├── app.py                 # Flask factory
│       ├── db.py                  # DB connection helpers
│       ├── api/                   # REST API blueprints
│       └── services/              # Business logic services
├── sql/warehouse/                 # SQL bootstrap files (run in order)
│   ├── 01_roles_and_schemas.sql
│   ├── 02_fdw_master_access.sql
│   ├── 03_inventory_dimensions.sql
│   ├── 04_inventory_fact_and_staging.sql
│   ├── 05_inventory_loads_and_views.sql
│   ├── 06_reorder_and_balance.sql
│   ├── 07_reorder_functions.sql
│   ├── 08_procurement_warehouse.sql
│   ├── 09_sales_warehouse.sql
│   └── 10_analytical_domains.sql
```

```
├── schemas/kafka/                # Kafka topic schemas (documentation)
│   ├── inventory_events.json
│   ├── procurement_events.json
│   └── sales_events.json
├── airflow/dags/                 # Airflow DAG definitions
│   ├── inventory_gold_pipeline.py
│   ├── procurement_gold_pipeline.py
│   └── sales_gold_pipeline.py
├── tests/                        # Test suite
├── docs/                         # Documentation
├── jars/                         # PostgreSQL JDBC driver
├── scripts/                      # Shell scripts for local dev
├── pyproject.toml                # Package definition + optional dependencies
├── requirements.txt              # Full lockfile for Docker/Airflow
└── .env.example                  # Environment template
```

Quick Setup

```
# 1. Clone the repository
git clone <repo-url>
cd Event-Driven-Inventory-Analytics-Warehouse-BI-System

# 2. Create virtual environment
python -m venv .venv
source .venv/bin/activate

# 3. Install core dependencies
pip install flask sqlparse psycopg2-binary confluent-kafka pyspark pytest

# 4. Install in editable mode (optional, for IDE support)
pip install -e ".[spark,dev]"

# 5. Set PYTHONPATH
export PYTHONPATH="$PWD/src"

# 6. Run tests (no infrastructure needed)
pytest tests/
```

Running Tests

The test suite is split into tiers:

Unit tests (no infrastructure required)

```
pytest tests/ -v
# 82 tests — no live database, Kafka, or Spark required
```

These tests cover:

File	Coverage
`test_cli.py`	CLI argument parsing and dry-run producer
`test_config.py`	Settings construction and credential validation
`test_contracts.py`	Inventory event building and all validation rules
`test_domain_contracts.py`	Procurement and sales contract validation
`test_platform.py`	StatusService, ControlPlaneStore, Flask routes
`test_sql_runner.py`	SQL template-variable rendering
`test_warehouse_functions.py`	Warehouse wrappers, auth middleware, credential warnings
`test_webapp_services.py`	Stock service (no `::uuid` casts), IdempotencyStore (TTL eviction + thread safety), TTLCache stampede protection, order state machine, reorder SET clause

Integration tests (requires live PostgreSQL)

```
MW_INTEGRATION_TEST=1 pytest tests/ -m integration -v
```

These tests connect to the actual database and verify stored procedure behaviour.

Key Design Patterns

Configuration

All settings come from environment variables via `src/medwarehouse/config.py`. The `get_settings()` function is `@lru_cache` — it reads env vars once per process and is cleared in tests via `get_settings.cache_clear()`. Weak credentials log a warning in `local`/`dev` environments and raise `RuntimeError` in all others.

CLI command structure

```
python -m medwarehouse <group> <subcommand> [options]

Groups: produce | spark | warehouse | orchestration | platform
```

Heavy imports (pyspark, psycopg2, confluent_kafka) are deferred to inside command handlers, so `import medwarehouse.cli` succeeds without those packages installed.

Event contract pattern

Each domain has a `contracts/<domain>.py` file that defines: - `build_<domain>_event(...)` — constructs a valid event dict - `validate_<domain>_event_dict(event)` — returns a list of error strings - `generate_<domain>_sample_events(...)` — deterministic sample data for testing

The Silver Spark job enforces the same rules as the contract validator, applied at scale.

Adding a new domain

1. Define the Kafka schema in `schemas/kafka/<domain>_events.json`
2. Create `src/medwarehouse/contracts/<domain>.py`
3. Update `src/medwarehouse/producers/<domain>.py`
4. Create Bronze/Silver/Stage Spark jobs following the existing pattern
5. Create `sql/warehouse/NN_<domain>_warehouse.sql`
6. Create `src/medwarehouse/warehouse/<domain>.py`
7. Add new paths to `PathSettings` in `config.py`
8. Add CLI subcommands to `cli.py`
9. Add `JobSpec` entries to `platform/control/catalog.py`
10. Create `airflow/dags/<domain>_gold_pipeline.py`
11. Add tests to `tests/test_domain_contracts.py`

Database access

- All DB access goes through `medwarehouse.warehouse.postgres.connect(connection)` — a context manager that handles rollback on exception and always closes the connection.
- SQL template substitution uses `render_sql_template(template, context)` which replaces `{{KEY}}` placeholders.
- Multi-statement SQL files are split using `sqlparse.split()` before execution.

API key authentication

Both Flask apps use `medwarehouse.platform.auth.register_api_key_auth(app, api_key, service=...)`. The `/health` endpoint and `/static/*` paths are always exempt. Keys are read from `MW_WEBAPP_API_KEY` and `MW_PLATFORM_API_KEY` environment variables.

Common Development Tasks

Add a new warehouse SQL object

Add SQL to the appropriate `sql/warehouse/NN_*.sql` file, then re-run `python -m medwarehouse warehouse bootstrap` to apply.

Add a new alert rule

1. Create a class in `src/medwarehouse/platform/services/alerts.py` inheriting `AlertRule`
2. Implement `evaluate(self, metrics: dict) -> Alert | None`
3. Add an instance to the `rules` list in `build_default_alert_engine()`

Add a new monitoring probe

1. Create a class in `src/medwarehouse/platform/probes/` inheriting `Probe`
2. Implement `_collect(self) -> dict`
3. Register it in `StatusService.__init__._probes` in `services/status.py`

Debug a Silver quarantine

Check the quarantine Parquet:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local").getOrCreate()
df = spark.read.parquet("data/quarantine/inventory_events/")
df.select("event_type", "quarantine_reasons").show(truncate=False)
```

Reset the warehouse

```
# Drop and recreate analytics schema
psql -U postgres -d medwarehouse_analytics -c "DROP SCHEMA analytics CASCADE; CREATE SCHEMA
        analytics;"
python -m medwarehouse warehouse bootstrap
```

Security Model

Authentication

API Key Authentication

Both services use static API key authentication via HTTP headers.

Header formats accepted:

```
X-API-Key: <key>
Authorization: Bearer <key>
```

Configuration:

```
MW_WEBAPP_API_KEY=<hex-32-bytes>     # Operator webapp (port 8080)
MW_PLATFORM_API_KEY=<hex-32-bytes>   # Monitoring platform (port 8787)
```

Generate a key:

```
openssl rand -hex 32
```

Constant-time comparison: Keys are compared using `hmac.compare_digest()` rather than the `==` operator. This prevents timing-based side-channel attacks where an attacker could infer the key length or prefix by measuring response latency differences.

Exemptions: The `/health` endpoint and `/static/*` paths are always accessible without authentication.

Behaviour when no key is configured: Auth is skipped entirely and a `SECURITY WARNING` is logged at startup. This is acceptable for local development only.

Database Role Model

Four PostgreSQL roles are used, following least-privilege principles:

Role	Database	Privileges	Used By
`postgres` (superuser)	Both	Unrestricted	Bootstrap scripts only
`spark_writer`	analytics	SELECT/INSERT/TRUNCATE on staging, INSERT on facts, EXECUTE on load functions	Spark jobs, webapp API calls that trigger pipeline
`analytics_reader`	analytics	SELECT on views, EXECUTE on quality check functions	Power BI, BI tools, read-only API
`fdw_reader`	master	SELECT on all exposed tables	FDW server user mapping

The `analytics_reader` role **cannot** write any data. It cannot access raw fact tables directly — only through views. It cannot access the master database tables directly.

The `fdw_reader` role on the master database has `SELECT` only. It has no INSERT, UPDATE, or DELETE privileges.

Credential Management

Default credentials (development only): All default credentials are documented in `.env.example`. They are intentionally weak (e.g., `postgres`/`postgres`) for local setup convenience.

Production enforcement: The `get_settings()` function in `config.py` calls `_warn_weak_credentials()` at startup: - In `local` / `dev` / `development` / `test` environments: logs a `WARNING` and continues - In all other environments: raises a `RuntimeError` and prevents startup

Credential rotation: Credentials are set in the `.env` file and passed to PostgreSQL at bootstrap via SQL template substitution. To rotate: 1. Update the relevant `MW_*_PASSWORD` variable in `.env` 2. Run `python -m medwarehouse warehouse bootstrap` to update the role's password in PostgreSQL

Sensitive Data Handling

Password hashes: The `users.password_hash` column in the master database is **never** exposed through analytics views. The `v_audit_activity` view uses `master.users` but explicitly excludes `password_hash`.

Bank account numbers: `supplier_bank_details` is not exposed in any analytics FDW view. It remains only in the OLTP system.

Government IDs / PII: The `customers.government_id` field is available in `v_customer_summary` because it is needed for controlled substance purchase verification. Grant the `analytics_reader` role only to users who are authorised to see customer PII.

Database dump: The `meddata_full_20250916_175921.dump` file in the repository root is an AI-generated sample dataset. It is listed in `.gitignore`. Do not commit real operational data dumps to version control.

Regulatory Compliance

This system handles pharmaceutical inventory, which is subject to Indian drug regulations:

Schedule H, H1, X, and NDPS drugs: - Sales must be linked to a valid prescription (`sales.prescription_id IS NOT NULL`) - The `v_prescription_compliance_violations` view surfaces any sales of these drugs without a prescription - The `ControlledSubstanceComplianceRule` alert fires if any violations are detected - The `v_controlled_substance_register` view provides the dispensing register required by law

Supplier drug licenses: - No procurement from suppliers with an expired `drug_license_no` - The `v_supplier_license_expiry` view flags EXPIRED, CRITICAL (≤30 days), and WARNING (≤90 days) licenses - The `SupplierLicenseExpiryRule` alert fires automatically when licenses approach expiry - The alert fires as CRITICAL when a license is expired or within 30 days

Audit trails: - All entity changes are recorded in `audit_logs` in the master database - The `v_audit_activity` analytics view provides analyst-safe access (no password data) - Audit logs are immutable — the analytics system does not modify them

---

Network Security

Kafka: The local Kafka broker uses plaintext listeners with no authentication. In a production deployment, configure SASL/SCRAM or mTLS.

PostgreSQL: In the Docker setup, PostgreSQL is bound to all interfaces (`0.0.0.0:5432`). For production, restrict to specific IP ranges using `pg_hba.conf`.

Flask services: Both the monitoring platform and operator webapp default to `127.0.0.1`. Use `--host 0.0.0.0` only behind a reverse proxy (nginx, Traefik) that handles TLS termination.

Known Security Gaps (Tracked)

Gap	Severity	Status	Mitigation
No TLS on Kafka	Medium	Open	Acceptable for internal/single-node deployments; configure SASL/SCRAM or mTLS for multi-node
No session-based auth (stateless API key only)	Low	Open	Add JWT or session tokens if per-user auditing is required
Airflow default credentials	High	**Configurable**	Set `AIRFLOW_ADMIN_PASSWORD` in `.env` before first `docker compose up`; also set `AIRFLOW__WEBSERVER__SECRET_KEY` for production
No rate limiting on Flask APIs	Low	Open	Add `Flask-Limiter` for production deployments
Flask pinned to 2.x in Airflow container	Low	Open	Upgrade Airflow image to 3.x when dependency constraints allow
Idempotency store is in-memory	Low	Open	Current `IdempotencyStore` uses a per-process dict with background TTL eviction; replace with Redis for multi-process or multi-instance deployments
PostgreSQL binds to `0.0.0.0` in Docker	Medium	Open	Restrict to specific IP ranges via `pg_hba.conf` or Docker network isolation for non-development deployments

---

Operator Webapp — API Reference

Base URL: `http://localhost:8080`

All endpoints require `X-API-Key: <key>` or `Authorization: Bearer <key>` unless `MW_WEBAPP_API_KEY` is unset.

The `/health` endpoint is always accessible.

> **ID types:** `product_id`, `warehouse_id`, `supplier_id`, `customer_id` are **text strings** (business keys), not UUID type. In the sample dataset they take the form `P-LOCAL-CALPOL-500` / `W-LOCAL-RECEIVING-01`. In a production OLTP deployment they may be UUID-format strings; do not apply PostgreSQL `::uuid` casts.

Health

`GET /health`

Returns service status. No authentication required.

Response 200:

```
{ "status": "ok", "service": "medwarehouse-webapp" }
```

Stock Entry (`/api/v1/stock`)

`POST /api/v1/stock/received`

Record goods received into a warehouse lot and emit a `STOCK_RECEIVED` Kafka event.

Request body:

```
{
  "product_id": "string",
  "warehouse_id": "string",
  "batch_number": "string",
  "expiry_date": "YYYY-MM-DD",
  "quantity": 200,
  "supplier_id": "string",          // optional
  "cost_price_per_unit": 85.50,     // optional
  "sale_price_per_unit": 120.00     // optional
}
```

Response 201:

```
{ "event_id": "uuid", "event_type": "STOCK_RECEIVED", "status": "accepted" }
```

Response 400: Missing required field. **Response 422:** Contract validation failure (e.g. quantity ≤ 0).

`POST /api/v1/stock/sold`

Record a stock sale, reduce lot quantity, and emit `STOCK_SOLD`.

Request body:

```
{
  "product_id": "string",
  "warehouse_id": "string",
  "batch_number": "string",
  "expiry_date": "YYYY-MM-DD",
  "quantity": 24,
  "sale_id": "string"
}
```

Response 201: `{ "event_id": ..., "event_type": "STOCK_SOLD", "status": "accepted" }` **Response 422:** Insufficient stock or lot not found.

`POST /api/v1/stock/adjusted`

Record a manual stock adjustment and emit `STOCK_ADJUSTED`.

Request body:

```
{
  "product_id": "string",
  "warehouse_id": "string",
  "batch_number": "string",
  "quantity_delta": -5,
  "reason": "AUDIT"
}
```

`reason` must be one of: `AUDIT`, `DAMAGE`, `SYSTEM_CORRECTION`

Response 201: `{ "event_id": ..., "event_type": "STOCK_ADJUSTED", "status": "accepted" }` **Response 422:** Lot not found or quantity_delta is zero.

`POST /api/v1/stock/expired`

Record expired stock removal and emit `STOCK_EXPIRED`.

Request body:

```
{
  "product_id": "string",
  "warehouse_id": "string",
  "batch_number": "string",
  "expiry_date": "YYYY-MM-DD",
  "quantity": 5
}
```

Response 201: `{ "event_id": ..., "event_type": "STOCK_EXPIRED", "status": "accepted" }`

`GET /api/v1/stock/movements`

List recent stock movement events from the analytics staging table.

Query params: - `limit` (int, default 50, max 500)

Response 200: Array of event objects with fields: `event_id`, `event_type`, `event_time`, `product_id`, `warehouse_id`, `batch_number`, `quantity_delta`, `supplier_id`, `sale_id`, `adjustment_reason`

Reference Data (`/api/v1`)

`GET /api/v1/products`

List all active products from the master database.

Response 200: Array of `{ product_id, brand_name, generic_name, form_factor, hsn_code, supplier_id }`

`GET /api/v1/suppliers`

List all active suppliers.

Response 200: Array of `{ supplier_id, name, gstin }`

`GET /api/v1/warehouses`

List all active warehouse locations.

Response 200: Array of `{ warehouse_id, warehouse_name, temperature_range }`

`GET /api/v1/inventory`

Current stock levels from `analytics.v_inventory_snapshot` (non-zero balances only).

Response 200: Array of `{ product_id, brand_name, generic_name, warehouse_id, warehouse_name, batch_number, expiry_date, current_quantity, last_event_time }`

`GET /api/v1/inventory/risk`

Products at or approaching reorder threshold, ordered by severity.

Response 200: Array of `{ product_id, brand_name, warehouse_id, warehouse_name, batch_number, expiry_date, current_quantity, reorder_point, quantity_gap, risk_status, lead_time_days }`

`risk_status` values: `OUT_OF_STOCK`, `BELOW_THRESHOLD`, `APPROACHING_THRESHOLD`, `HEALTHY`

Reorder Policies (`/api/v1/reorder-policies`)

`GET /api/v1/reorder-policies`

List all active reorder policies.

Response 200: Array of `{ id, product_id, warehouse_id, reorder_point, reorder_quantity, preferred_supplier_id, lead_time_days, active, created_at, updated_at }`

`POST /api/v1/reorder-policies`

Create a reorder policy. Replaces any existing active policy for the same product+warehouse.

Request body:

```
{
  "product_id": "string",
  "warehouse_id": "string",
  "reorder_point": 50,
  "reorder_quantity": 200,
  "preferred_supplier_id": "string",  // optional
  "lead_time_days": 7                 // optional, default 7
}
```

Response 201: `{ "id": 42, "status": "created" }`

`PUT /api/v1/reorder-policies/{id}`

Update an existing policy's thresholds.

Request body: Any subset of `{ reorder_point, reorder_quantity, preferred_supplier_id, lead_time_days }`

Response 200: `{ "id": 42, "status": "updated" }`

`DELETE /api/v1/reorder-policies/{id}`

Deactivate a policy (sets `active = FALSE`).

Response 200: `{ "id": 42, "status": "deactivated" }`

Purchase Orders (`/api/v1/orders`)

Auto-generated purchase orders flow: `DRAFT → APPROVED → SENT → RECEIVED / CANCELLED`

`GET /api/v1/orders`

List purchase orders with enriched product and supplier names.

Query params: - `status` (string, optional) — filter by status: `DRAFT`, `APPROVED`, `SENT`, `RECEIVED`, `CANCELLED`

Response 200: Array from `v_purchase_order_status`

`POST /api/v1/orders/{po_id}/approve`

Approve a DRAFT purchase order.

Request body:

```
{ "approved_by": "manager_username" }
```

Response 200: `{ "po_id": "...", "status": "approved" }` **Response 200 (not found):** `{ "po_id": "...", "status": "not_found_or_wrong_state" }`

`POST /api/v1/orders/{po_id}/send`

Mark an APPROVED order as SENT and dispatch supplier email (if SMTP is configured).

Response 200: `{ "po_id": "...", "status": "sent" }`

`POST /api/v1/orders/{po_id}/cancel`

Cancel a DRAFT or APPROVED order.

Response 200: `{ "po_id": "...", "status": "cancelled" }`

`POST /api/v1/orders/{po_id}/receive`

Mark a SENT order as RECEIVED and emit a `STOCK_RECEIVED` Kafka event to update inventory.

Response 200: `{ "po_id": "...", "status": "received", "stock_event_id": "uuid" }`

Monitoring Platform API Reference

Base URL: `http://localhost:8787/api`

All endpoints support `?refresh=hard` to bypass the 60-second status cache.

Endpoint	Description
`GET /api/status`	Legacy snapshot: environment, jobs, infra, alerts
`GET /api/status/full`	Complete status including all probe metrics
`GET /api/status/pipeline`	Pipeline freshness metrics and alerts
`GET /api/status/infra`	Infrastructure service status and alerts
`GET /api/status/alerts`	Active and recent alert list
`POST /api/jobs/{job_id}/start`	Start a registered pipeline job
`POST /api/jobs/{job_id}/stop`	Stop a running long-running job
`POST /api/infra/start`	Start infrastructure (docker-compose up)
`POST /api/infra/stop`	Stop infrastructure (docker-compose down)

Available `job_id` values: `warehouse_bootstrap`, `inventory_producer`, `inventory_bronze`, `inventory_silver`, `inventory_stage`, `refresh_dimensions`, `load_inventory_facts`, `refresh_views`, `quality_checks`, `build_gold`, `refresh_balance`, `check_reorders`, `procurement_producer`, `procurement_bronze`, `procurement_silver`, `build_procurement_gold`, `sales_producer`, `sales_bronze`, `sales_silver`, `build_sales_gold`

Runbook

Operational guide for running, monitoring, and recovering all three domain pipelines.

Quick Start (all three pipelines)

```
# 1. Configure environment
cp .env.example .env && $EDITOR .env

# 2. Start infrastructure
./scripts/start_services.sh

# 3. Bootstrap analytics warehouse (run once)
export PYTHONPATH=src
python -m medwarehouse warehouse bootstrap

# 4. Start monitoring platform
python -m medwarehouse platform serve --host 0.0.0.0 --port 8787

# 5. Start operator webapp
python -m medwarehouse_webapp --host 0.0.0.0 --port 8080
```

Local Environment

Set environment variables by sourcing `.env`:

```
set -a && source .env && set +a
export PYTHONPATH=src
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Inventory Pipeline

```
# Produce sample events
python -m medwarehouse produce inventory --max-events 20

# Start Bronze stream (long-running, separate terminal)
python -m medwarehouse spark inventory-bronze

# After events land in Bronze:
python -m medwarehouse spark inventory-silver
python -m medwarehouse orchestration build-gold

# Or use the convenience script:
./scripts/run_inventory_flow.sh 20
```

build-gold runs: validate-silver → stage-inventory-events → refresh-dimensions → load-facts → refresh-views → quality-checks → refresh-balance → check-reorders

---

Procurement Pipeline

```
python -m medwarehouse produce procurement --max-events 15

# Start Bronze stream (separate terminal)
python -m medwarehouse spark procurement-bronze

python -m medwarehouse spark procurement-silver
python -m medwarehouse orchestration build-procurement-gold

# Or use:
./scripts/run_procurement_flow.sh 15
```

build-procurement-gold runs: validate-silver → stage-procurement-events → refresh-dimensions → load-facts → refresh-views → quality-checks

---

Sales Pipeline

```
python -m medwarehouse produce sales --max-events 20

# Start Bronze stream (separate terminal)
python -m medwarehouse spark sales-bronze

python -m medwarehouse spark sales-silver
python -m medwarehouse orchestration build-sales-gold

# Or use:
./scripts/run_sales_flow.sh 20
```

Airflow

Three DAGs are available (`schedule=None` — trigger manually):

```
http://localhost:8090
DAGs: inventory_gold_pipeline | procurement_gold_pipeline | sales_gold_pipeline
Default credentials: admin / admin (change AIRFLOW_ADMIN_PASSWORD in .env before first start)
```

> Airflow webserver runs on port **8090** (remapped from default 8080 to avoid conflict with the operator webapp on 8080).

Bootstrap Replay

If the warehouse schema is corrupted or needs to be rebuilt:

```
# Drop and recreate analytics schema (DESTRUCTIVE — all analytics data lost)
psql -U postgres -d medwarehouse_analytics -c "DROP SCHEMA analytics CASCADE; CREATE SCHEMA
        analytics;"
python -m medwarehouse warehouse bootstrap
```

Quality Checks

```
python -m medwarehouse warehouse quality-checks
python -m medwarehouse warehouse procurement-quality-checks
python -m medwarehouse warehouse sales-quality-checks
```

Each command raises on any failing assertion and exits non-zero.

Reorder Automation

```
# Refresh the physical inventory snapshot
python -m medwarehouse warehouse refresh-balance

# Check stock against reorder policies → creates DRAFT purchase orders
python -m medwarehouse warehouse check-reorders

# View draft orders in the webapp or query directly:
# SELECT * FROM analytics.v_purchase_order_status WHERE status = 'DRAFT';
```

Silver Quarantine Investigation

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local").getOrCreate()

# Inventory
df = spark.read.parquet("data/quarantine/inventory_events/")
df.select("event_type", "quarantine_reasons").show(truncate=False)

# Procurement
df = spark.read.parquet("data/quarantine/procurement_events/")
df.select("event_type", "quarantine_reasons").show(truncate=False)
```

Recovery: Bronze Stream Stuck

1. Kill the stuck streaming job via `POST /api/jobs/inventory_bronze/stop` or Ctrl+C
2. Delete checkpoints: `rm -rf data/checkpoints/inventory_events/`
3. Restart Bronze: `python -m medwarehouse spark inventory-bronze --starting-offsets latest`

Infrastructure

```
./scripts/start_services.sh           # Start PostgreSQL + Kafka
./scripts/start_services.sh --with-airflow  # Include Airflow
./scripts/stop_services.sh            # Stop all services
```

Health check:

```
curl http://localhost:8787/health   # Monitoring platform
curl http://localhost:8080/health   # Operator webapp
```

Compliance Checks

```
-- Controlled substance violations (schedule H1/X/NDPS dispensed without prescription)
SELECT * FROM analytics.v_prescription_compliance_violations;

-- Supplier drug licenses expiring soon
SELECT * FROM analytics.v_supplier_license_expiry WHERE license_status IN
        ('EXPIRED','CRITICAL','WARNING');

-- Sales of recalled/banned products
SELECT * FROM analytics.v_recalled_product_sales;
```

Data Dictionary

Complete reference for all tables, views, and functions in the medwarehouse system. Last updated: 2026-06-06

Source Database — `medwarehouse_master` (OLTP)

All tables live in the `public` schema. UUID primary keys are generated by `gen_random_uuid()`.

`users`

Operator accounts with role-based access.

Column	Type	Description
user_id	uuid PK	Unique user identifier
username	varchar(50) UNIQUE	Login handle
full_name	varchar(255)	Display name
email	varchar(255)	Contact email
password_hash	varchar(255)	Bcrypt hash — never exposed to analytics
role	user_role ENUM	`admin`, `staff`, `manager`
status	user_status ENUM	`active`, `inactive`, `suspended`
created_at	timestamptz	Account creation timestamp
last_login	timestamptz	Most recent successful login

`customers`

Retail and institutional buyers.

Column	Type	Description
customer_id	uuid PK	Unique customer identifier
full_name	varchar(255)	Customer’s full name
phone	varchar(20)	Contact phone
email	varchar(255)	Contact email
address	text	Free-text address
date_of_birth	date	DOB for age-restricted drugs
government_id	varchar(50)	National ID / Aadhaar — required for Schedule H1/X/NDPS purchases
created_at	timestamptz	Record creation timestamp

`suppliers`

Drug manufacturers, distributors, and wholesalers.

Column	Type	Description
supplier_id	uuid PK	Unique supplier identifier
name	varchar(255)	Legal entity name
gstin	varchar(15)	GST Identification Number
pan_number	varchar(10)	Permanent Account Number
drug_license_no	varchar(100)	State drug license number
license_expiry_date	date	**CRITICAL**: License must be valid for any procurement
supplier_type	varchar(50)	Manufacturer, Distributor, Wholesaler, etc.
payment_terms	varchar(50)	e.g. “Net 30 Days”
typical_lead_time_days	integer	Average delivery lead time
status	supplier_status ENUM	`active`, `inactive`
notes	text	Free-text notes
created_at	timestamptz	Record creation timestamp

`supplier_contacts`

Named contacts at each supplier.

Column	Type	Description
contact_id	uuid PK	Unique contact identifier
supplier_id	uuid FK → suppliers	Parent supplier
contact_name	varchar(255)	Contact’s name
role	varchar(50)	e.g. Sales Manager, Account Executive
phone	varchar(20)	Direct phone
email	varchar(255)	Direct email
is_primary	boolean	One primary contact per supplier

`supplier_addresses`

Registered, billing, and shipping addresses per supplier.

Column	Type	Description
address_id	uuid PK	Unique address identifier
supplier_id	uuid FK → suppliers	Parent supplier
type	address_type ENUM	`billing`, `shipping`, `registered`
address_line_1	text	Street address
city	varchar(100)	City
state	varchar(100)	State
pincode	varchar(10)	Postal code

`supplier_bank_details`

Bank accounts for supplier payments.

Column	Type	Description
bank_detail_id	uuid PK	Unique bank detail identifier
supplier_id	uuid FK → suppliers	Parent supplier
bank_name	varchar(255)	Bank name
account_holder_name	varchar(255)	Account holder name
account_number	varchar(50)	Bank account number
ifsc_code	varchar(11)	IFSC code (Indian Financial System Code)
is_default	boolean	Primary payment account

`products`

Drug and pharmaceutical product catalog.

Column	Type	Description
product_id	uuid PK	Unique product identifier
supplier_id	uuid FK → suppliers	Default/primary supplier
brand_name	varchar(255)	Brand/trade name
generic_name	varchar(255)	INN / generic name
manufacturer	varchar(255)	Manufacturing company
hsn_code	varchar(8)	HSN code for GST classification
tax_slab_percentage	numeric(4,2)	Applicable GST %
mrp_per_unit	numeric(10,2)	Maximum retail price
strength_value	numeric(10,2)	Drug strength (e.g. 500 for 500mg)
strength_unit	varchar(20)	Unit (mg, ml, IU, etc.)
form_factor	product_form ENUM	Tablet, Capsule, Syrup, Injection, Ointment, Powder, Drops, Cream, Gel, Inhaler
units_per_strip	integer	Packaging info
strips_per_case	integer	Packaging info
reorder_level	integer	Default minimum stock before reorder
drug_schedule	schedule_type ENUM	`OTC`, `Schedule G`, `Schedule H`, `Schedule H1`, `Schedule X`, `NDPS`, `Generic`, `Other`
therapeutic_class	varchar(100)	ATC classification
status	product_status ENUM	`active`, `discontinued`, `banned`, `recalled`
created_at	timestamptz	Record creation timestamp

`product_barcodes`

Multiple barcodes per product (EAN-13, QR, etc.).

Column	Type	Description
barcode_id	uuid PK	Unique barcode identifier
product_id	uuid FK → products	Parent product
barcode	varchar(50) UNIQUE	Barcode value
type	varchar(20)	EAN-13, QR, DataMatrix, etc.

`warehouse_locations`

Physical storage locations.

Column	Type	Description
location_id	uuid PK	Unique location identifier
name	varchar(100)	Location name
description	text	Free-text description
temperature_range	varchar(50)	e.g. “2°C – 8°C” (cold chain), “15°C – 25°C”
is_active	boolean	Active/decommissioned

`inventory_lots`

Physical batch/lot quantity ledger. This is the authoritative source of current stock.

Column	Type	Description
lot_id	uuid PK	Unique lot identifier
product_id	uuid FK → products	Product this lot belongs to
location_id	uuid FK → warehouse_locations	Storage location
batch_number	varchar(100)	Manufacturer batch/lot number
quantity_on_hand	integer (≥ 0)	Current physical quantity
cost_price_per_unit	numeric(10,2)	Purchase cost
sale_price_per_unit	numeric(10,2)	Current selling price
manufactured_date	date	Manufacturing date
expiry_date	date NOT NULL	Expiry date
received_at	timestamptz	When this lot was received into the warehouse

`stock_adjustments`

Manual stock corrections and write-offs.

Column	Type	Description
adjustment_id	uuid PK	Unique adjustment identifier
lot_id	uuid FK → inventory_lots	Lot being adjusted
adjusted_by	uuid FK → users	User who made the adjustment
adjustment_date	timestamptz	When the adjustment was recorded
adjustment_type	varchar(50)	MANUAL, EXPIRY, DAMAGE, AUDIT, SYSTEM_CORRECTION
quantity_changed	integer	Net quantity change (positive or negative)
remarks	text	Reason / notes

`purchase_orders`

Purchase order headers.

Column	Type	Description
po_id	uuid PK	Unique PO identifier
supplier_id	uuid FK → suppliers	Supplier being ordered from
created_by	uuid FK → users	User who created the PO
po_number	varchar(50) UNIQUE	Human-readable PO reference
order_date	date	Date PO was created
expected_delivery_date	date	Agreed delivery date
status	varchar(50)	pending, approved, received, cancelled
notes	text	Free-text notes

`purchase_order_items`

Purchase order line items.

Column	Type	Description
po_item_id	uuid PK	Unique line item identifier
po_id	uuid FK → purchase_orders	Parent PO
product_id	uuid FK → products	Product ordered
quantity_ordered	integer (> 0)	Quantity ordered
unit_price	numeric(10,2)	Agreed unit price
line_total	numeric(12,2) GENERATED	`quantity_ordered × unit_price`

`goods_receipts`

Goods Receipt Note (GRN) headers — recorded when delivery arrives.

Column	Type	Description
grn_id	uuid PK	Unique GRN identifier
po_id	uuid FK → purchase_orders	Parent PO
received_by	uuid FK → users	User who received the delivery
receipt_date	timestamptz	Date and time of receipt
status	varchar(50)	received, partial, rejected
remarks	text	Notes on delivery condition

`goods_receipt_items`

GRN line items — actual quantities received per lot.

Column	Type	Description
grn_item_id	uuid PK	Unique GRN line item identifier
grn_id	uuid FK → goods_receipts	Parent GRN
lot_id	uuid FK → inventory_lots	Inventory lot created/updated
product_id	uuid FK → products	Product received
quantity_received	integer (≥ 0)	Actual quantity received
unit_cost	numeric(10,2)	Actual unit cost on this receipt
expiry_date	date	Expiry date on the physical pack
batch_number	varchar(100)	Batch number on the physical pack

`sales`

Sale / invoice headers.

Column	Type	Description
sale_id	uuid PK	Unique sale identifier
user_id	uuid FK → users	Staff who processed the sale
customer_id	uuid FK → customers	Buyer (nullable for walk-in)
prescription_id	uuid FK → prescriptions	Linked prescription (required for Schedule H+)
invoice_number	varchar(50) UNIQUE	Human-readable invoice reference
subtotal_amount	numeric(12,2)	Sum of line totals before discount/tax
total_discount_amount	numeric(12,2)	Total discount applied
total_tax_amount	numeric(12,2)	Total GST collected
total_amount_payable	numeric(12,2)	Final amount charged to customer
payment_method	payment_method ENUM	Cash, UPI, Credit Card, Debit Card, Bank Transfer, Cheque
status	sale_status ENUM	`completed`, `returned`, `cancelled`
sale_timestamp	timestamptz	Sale date and time
notes	text	Free-text notes

`sale_items`

Sale line items.

Column	Type	Description
sale_item_id	uuid PK	Unique line item identifier
sale_id	uuid FK → sales	Parent sale
lot_id	uuid FK → inventory_lots	Lot consumed by this sale
product_id	uuid FK → products	Product sold
quantity_sold	integer (> 0)	Quantity sold
mrp_at_sale	numeric(10,2)	MRP at the time of sale
price_per_unit_at_sale	numeric(10,2)	Actual selling price
discount_amount	numeric(10,2)	Discount on this line
tax_percentage_at_sale	numeric(4,2)	GST % applied
line_total	numeric(12,2)	Net line total after discount

`prescriptions`

Prescription records for regulated drugs.

Column	Type	Description
prescription_id	uuid PK	Unique prescription identifier
customer_id	uuid FK → customers	Patient
doctor_name	varchar(255)	Prescribing doctor’s name
prescription_date	date	Date on the prescription
file_path	text	Path to scanned prescription image
notes	text	Additional notes

`prescription_items`

Line items on a prescription.

Column	Type	Description
prescription_item_id	uuid PK	Unique line item identifier
prescription_id	uuid FK → prescriptions	Parent prescription
product_id	uuid FK → products	Prescribed drug
dosage_instructions	text	Dosage and frequency

`payments`

Payment records for both customer sales and supplier invoices.

Column	Type	Description
payment_id	uuid PK	Unique payment identifier
related_sale_id	uuid FK → sales	Linked sale (nullable if AP payment)
related_po_id	uuid FK → purchase_orders	Linked PO (nullable if AR payment)
payer_type	varchar(20)	`customer` or `supplier`
payment_method	payment_method ENUM	Payment mode used
amount	numeric(12,2) (> 0)	Payment amount
payment_date	timestamptz	Date and time of payment
transaction_ref	varchar(100)	Bank/UPI transaction reference
status	varchar(50)	completed, pending, failed, reversed

`audit_logs`

Immutable change log for all entity mutations.

Column	Type	Description
log_id	bigint PK (auto-increment)	Sequential log identifier
user_id	uuid FK → users	User who made the change
entity_id	uuid	ID of the modified record
action	varchar(100)	INSERT, UPDATE, DELETE, LOGIN, LOGOUT
entity	varchar(100)	Table name
old_values	jsonb	Previous state (UPDATE/DELETE)
new_values	jsonb	New state (INSERT/UPDATE)
timestamp	timestamptz	When the change occurred
ip_address	varchar(50)	Client IP address

Analytics Database — `medwarehouse_analytics`

Schema: `master` (Foreign Data Wrapper)

Read-only aliases to the source database tables via postgres_fdw. Tables: `products`, `suppliers`, `warehouse_locations`, `goods_receipts`, `goods_receipt_items`, `stock_adjustments`, `sales`, `sale_items`, `purchase_orders`, `purchase_order_items`, `prescriptions`, `prescription_items`, `payments`, `customers`, `users`, `audit_logs`

Schema: `analytics` — Dimensions (SCD Type 2)

`DIM_PRODUCT`

Column	Type	Description
product_sk	bigserial PK	Surrogate key
product_id	text	Business key (UUID from master)
supplier_id	text	Supplier business key
brand_name	text	Brand/trade name
generic_name	text	Generic/INN name
form_factor	text	Product form
hsn_code	text	HSN code
valid_from	timestamptz	SCD2 validity start
valid_to	timestamptz	SCD2 validity end (NULL = current)
is_current	boolean	TRUE for current version
created_at / updated_at	timestamptz	Record timestamps

`DIM_SUPPLIER`

Column	Type	Description
supplier_sk	bigserial PK	Surrogate key
supplier_id	text	Business key
supplier_name	text	Supplier name
gstin	text	GST number
valid_from / valid_to / is_current	—	SCD2 fields

`DIM_WAREHOUSE`

Column	Type	Description
warehouse_sk	bigserial PK	Surrogate key
warehouse_id	text	Business key (location_id from master)
warehouse_name	text	Location name
temperature_range	text	Storage temperature
valid_from / valid_to / is_current	—	SCD2 fields

`DIM_CUSTOMER`

Column	Type	Description
customer_sk	bigserial PK	Surrogate key
customer_id	text UNIQUE	Customer type grouping key
full_name	text	Customer type label
customer_type	text	RETAIL, HOSPITAL, DISTRIBUTOR
created_at / updated_at	timestamptz	Record timestamps

Schema: `analytics` — Staging Tables

Table	Domain	Purpose
`stg_inventory_events`	Inventory	Spark JDBC landing zone for validated inventory events
`stg_procurement_events`	Procurement	Spark JDBC landing zone for validated procurement events
`stg_sales_events`	Sales	Spark JDBC landing zone for validated sales events

All staging tables are truncate-and-load. They hold the most recent pipeline run only.

Schema: `analytics` — Fact Tables

`FACT_INVENTORY_EVENTS`

Partitioned by `event_time` (monthly RANGE). One row per `STOCK_RECEIVED`, `STOCK_SOLD`, `STOCK_ADJUSTED`, or `STOCK_EXPIRED` event. Append-only; never updated.

`FACT_PROCUREMENT_EVENTS`

Partitioned by `event_time` (monthly RANGE). One row per `PO_CREATED`, `PO_APPROVED`, `PO_RECEIVED`, or `PO_CANCELLED` event. Append-only.

`FACT_SALES_EVENTS`

Partitioned by `event_time` (monthly RANGE). One row per `SALE_CREATED` or `SALE_CANCELLED` event. Includes computed `line_revenue` (positive for sales, negative for cancellations). Append-only.

Schema: `analytics` — Operational Tables

Table	Purpose
`fact_inventory_balance`	Physical inventory snapshot per product+warehouse+batch. Refreshed after each inventory Gold run. Supports fast BI queries.
`reorder_policies`	Operator-configured reorder thresholds per product+warehouse.
`pending_purchase_orders`	Auto-generated draft POs awaiting operator approval. Lifecycle: DRAFT → APPROVED → SENT → RECEIVED / CANCELLED.

Schema: `analytics` — Views

INVENTORY

View	Description
`v_dim_product_current`	Current active product dimension rows
`v_dim_supplier_current`	Current active supplier dimension rows
`v_dim_warehouse_current`	Current active warehouse dimension rows
`v_inventory_balance`	Running SUM(quantity_delta) per product+warehouse+batch. Derived from fact_inventory_events.
`v_inventory_snapshot`	Enriched non-zero balances with product and warehouse names. Primary BI view for current stock.
`v_reorder_risk`	Products approaching or below their reorder threshold. Includes risk_status: OUT_OF_STOCK, BELOW_THRESHOLD, APPROACHING_THRESHOLD, HEALTHY.
`v_purchase_order_status`	Enriched pending_purchase_orders with product and supplier names.

PROCUREMENT

View	Description
`v_po_lifecycle`	Latest status per PO with full lifecycle timestamps (created → approved → received → cancelled).
`v_supplier_performance`	Per-supplier fulfillment rate, fill rate, average lead time, and open PO count.
`v_po_aging`	Open POs (PENDING or APPROVED) ordered by age in days.

SALES

View	Description
`v_revenue_summary`	Daily net revenue by product, warehouse, currency, and customer type.
`v_sales_velocity`	Daily units sold with 7-day and 30-day rolling averages per product+warehouse.
`v_returns_analysis`	Cancellation rate per product.

COMPLIANCE & PRESCRIPTIONS

View	Description
`v_prescription_fill_rate`	Prescription fulfillment rate — prescriptions linked to completed sales.
`v_controlled_substance_register`	All Schedule H1/X/NDPS drug dispensing records. Required regulatory register.
`v_prescription_compliance_violations`	Schedule H1/X/NDPS sales without a linked prescription — active violations.
`v_doctor_prescribing_patterns`	Drug prescribing volume per doctor.

SUPPLIER MANAGEMENT

View	Description
`v_supplier_license_expiry`	Active suppliers with license status: VALID, WARNING (≤90 days), CRITICAL (≤30 days), EXPIRED.
`v_supplier_directory`	Active suppliers with primary contact and shipping address.

FINANCIAL

View	Description
`v_accounts_payable`	Outstanding amounts due to suppliers per PO.
`v_payment_method_breakdown`	Monthly payment volume by method and payer type.
`v_daily_collections`	Daily customer payment totals by payment method.

CUSTOMER ANALYTICS

View	Description
`v_customer_summary`	Customer lifetime value, purchase frequency, first/last purchase date.

AUDIT & COMPLIANCE

View	Description
`v_audit_activity`	Audit log enriched with username and role. Excludes password_hash.
`v_recalled_product_sales`	Sales of recalled or banned products.

STAFF PERFORMANCE

View	Description
`v_staff_performance`	Per-user completed sales, returns processed, stock adjustments, GRNs received.
`v_inactive_user_accounts`	Active accounts with no login for > 90 days.

Schema: `analytics` — Stored Functions

Function	Returns	Description
`refresh_inventory_dimensions()`	BIGINT	SCD2 upsert for dim_product, dim_supplier, dim_warehouse
`load_inventory_event_facts()`	BIGINT	Insert new inventory events from staging to fact
`refresh_inventory_semantic_layer()`	VOID	Recreate all inventory views
`run_inventory_quality_checks()`	TABLE	7 assertions: null keys, duplicates, unloaded events, orphans, default partition
`refresh_inventory_balance()`	BIGINT	MERGE v_inventory_balance → fact_inventory_balance
`check_reorder_thresholds()`	BIGINT	Create DRAFT purchase orders for products below reorder_point
`load_procurement_event_facts()`	BIGINT	Insert new procurement events from staging to fact
`refresh_procurement_semantic_layer()`	VOID	Recreate all procurement views
`run_procurement_quality_checks()`	TABLE	4 assertions for procurement data quality
`refresh_dim_customer()`	BIGINT	Upsert customer type dimension
`load_sales_event_facts()`	BIGINT	Insert new sales events from staging to fact
`refresh_sales_semantic_layer()`	VOID	Recreate all sales views
`run_sales_quality_checks()`	TABLE	5 assertions for sales data quality
`ensure_*_fact_partition(DATE)`	VOID	Create monthly partition for a given month
`ensure_*_fact_partitions(TIMESTAMPTZ, TIMESTAMPTZ)`	VOID	Create all partitions in a date range

Domain: Inventory Management

Purpose

Track all physical stock movements for every product, warehouse, and batch in the system. This is the system of record for current stock levels and the full movement history.

Status

Fully implemented — end-to-end from Kafka to Power BI.

Event Types

Event	Trigger	Quantity
`STOCK_RECEIVED`	Goods received from supplier	Positive
`STOCK_SOLD`	Units sold/dispensed to customer	Negative
`STOCK_ADJUSTED`	Manual correction (audit, damage, system)	Any non-zero
`STOCK_EXPIRED`	Expired units removed from stock	Negative

Pipeline

```
Kafka: inventory_events
  → Bronze Parquet: data/bronze/inventory_events/
  → Silver Parquet: data/silver/inventory_events/  (validated, deduplicated)
  → Quarantine:     data/quarantine/inventory_events/  (failed validation)
  → stg_inventory_events
  → fact_inventory_events  (partitioned by event_time)
  → fact_inventory_balance (snapshot)
  → v_inventory_snapshot   (BI view)
```

Validation Rules (Silver)

- `event_id`, `event_type`, `event_time`, `producer` required
- `event_type` must be one of the 4 supported types
- `product_id`, `warehouse_id`, `batch_number`, `quantity_delta` required
- `expiry_date` required for STOCK_RECEIVED, STOCK_SOLD, STOCK_EXPIRED
- `STOCK_RECEIVED`: quantity > 0
- `STOCK_SOLD`, `STOCK_EXPIRED`: quantity < 0
- `STOCK_ADJUSTED`: quantity ≠ 0, `reason` required
- `STOCK_SOLD`: `sale_id` required
- Duplicate `event_id` values quarantined

Key Analytics Objects

- `v_inventory_snapshot` — current stock by product+warehouse+batch with names
- `v_inventory_balance` — derived balance (SUM of deltas)
- `fact_inventory_balance` — physical snapshot (fast queries)
- `v_reorder_risk` — products below threshold

Quality Checks

7 checks: null business keys, duplicate event IDs, staged events not loaded, null dimension keys, orphan product/warehouse dimension, rows in default partition.

CLI Commands

```
python -m medwarehouse produce inventory [--max-events N]
python -m medwarehouse spark inventory-bronze
python -m medwarehouse spark inventory-silver
python -m medwarehouse orchestration build-gold
python -m medwarehouse warehouse refresh-balance
python -m medwarehouse warehouse check-reorders
```

Airflow DAG

`inventory_gold_pipeline` — 8 tasks: validate_silver → stage → refresh_dimensions → load_facts → refresh_views → quality_checks → refresh_balance → check_reorders

---

Domain: Procurement Lifecycle

Purpose

Track the full lifecycle of purchase orders from creation through approval, receipt, and cancellation. Enables supplier performance analysis and procurement KPIs.

Status

Fully implemented — end-to-end from Kafka to analytics views.

Event Types

Event	Trigger	Required Fields
`PO_CREATED`	Purchase order raised	`po_id`, `product_id`, `ordered_quantity`, `supplier_id`, `trigger_reason`
`PO_APPROVED`	Manager approves PO	`po_id`, `approved_by`
`PO_RECEIVED`	Goods received against PO	`po_id`, `product_id`, `received_quantity`, `warehouse_id`
`PO_CANCELLED`	PO cancelled before receipt	`po_id`, `reason`

`trigger_reason` must be `AUTO_REORDER` or `MANUAL`.

PO Lifecycle State Machine

```
PO_CREATED → PO_APPROVED → PO_RECEIVED  (fulfilled)
PO_CREATED → PO_CANCELLED               (cancelled)
```

Pipeline

```
Kafka: procurement_events
  → Bronze Parquet: data/bronze/procurement_events/
  → Silver Parquet: data/silver/procurement_events/
  → Quarantine:     data/quarantine/procurement_events/
  → stg_procurement_events
  → fact_procurement_events  (partitioned by event_time)
  → v_po_lifecycle
  → v_supplier_performance
  → v_po_aging
```

Key Analytics Objects

- `v_po_lifecycle` — latest status per PO with all lifecycle timestamps and computed `actual_lead_time_days`
- `v_supplier_performance` — per-supplier KPIs: fulfillment rate, fill rate, average lead time
- `v_po_aging` — open POs (PENDING/APPROVED) ordered by age

Quality Checks

4 checks: null po_id, duplicate event IDs, staged events not loaded, rows in default partition.

Integration with Reorder System

When the automated reorder service (`check_reorder_thresholds`) creates a DRAFT purchase order in `pending_purchase_orders`, the operator reviews and approves it in the webapp. On approval and send, the operator records receipt via `POST /api/v1/orders/{po_id}/receive` — which emits a `STOCK_RECEIVED` inventory event and closes the procurement loop.

CLI Commands

```
python -m medwarehouse produce procurement [--max-events N]
python -m medwarehouse spark procurement-bronze
python -m medwarehouse spark procurement-silver
python -m medwarehouse orchestration build-procurement-gold
```

Airflow DAG

`procurement_gold_pipeline` — 6 tasks: validate_silver → stage → refresh_dimensions → load_facts → refresh_views → quality_checks

---

Domain: Sales & Revenue

Purpose

Track every sale and cancellation to provide real-time revenue visibility, demand forecasting, and staff performance measurement.

Status

Fully implemented — end-to-end from Kafka to analytics views.

Event Types

Event	Trigger	Required Fields
`SALE_CREATED`	Sale completed	`sale_id`, `product_id`, `warehouse_id`, `quantity`, `unit_price`, `currency`, `customer_type`
`SALE_CANCELLED`	Sale reversed	`sale_id`, `product_id`, `warehouse_id`, `quantity`, `original_event_id`

`customer_type` must be: `RETAIL`, `HOSPITAL`, or `DISTRIBUTOR`. `currency` must be: `INR`, `USD`, `EUR`, or `GBP`. `quantity` must be positive for both event types.

Pipeline

```
Kafka: sales_events
  → Bronze Parquet: data/bronze/sales_events/
  → Silver Parquet: data/silver/sales_events/
  → Quarantine:     data/quarantine/sales_events/
  → stg_sales_events
  → fact_sales_events  (partitioned by event_time, includes computed line_revenue)
  → v_revenue_summary
  → v_sales_velocity
  → v_returns_analysis
```

`line_revenue` is computed during fact load: `+quantity × unit_price` for SALE_CREATED, `-quantity × unit_price` for SALE_CANCELLED.

Key Analytics Objects

- `v_revenue_summary` — daily net revenue by product, warehouse, currency, and customer type
- `v_sales_velocity` — daily units with 7-day and 30-day rolling averages per product+warehouse
- `v_returns_analysis` — cancellation rate and return volume per product

Quality Checks

5 checks: null sale_id, duplicate event IDs, staged events not loaded, negative revenue on SALE_CREATED, rows in default partition.

Power BI Usage

Connect to `v_revenue_summary` for revenue dashboards. Use `v_sales_velocity` for demand forecasting and reorder trigger tuning. All views include product and warehouse names — no joins needed in Power BI.

CLI Commands

```
python -m medwarehouse produce sales [--max-events N]
python -m medwarehouse spark sales-bronze
python -m medwarehouse spark sales-silver
python -m medwarehouse orchestration build-sales-gold
```

Airflow DAG

`sales_gold_pipeline` — 6 tasks: validate_silver → stage → refresh_dimensions → load_facts → refresh_views → quality_checks

---

Domain: Compliance & Audit

Purpose

Provide immutable audit trails, detect regulatory violations, and identify security risks such as recalled product sales and inactive user accounts.

Status

Implemented as analytics views — data read from master DB via FDW.

Source Tables (via FDW)

- `master.audit_logs` — all entity change records
- `master.users` — user accounts with roles
- `master.products` — `drug_schedule` and `status` fields
- `master.sales` + `master.sale_items` — for product-level compliance checks

Key Analytics Objects

`v_audit_activity`

Audit log enriched with username and role. Excludes `password_hash`. Covers: INSERT, UPDATE, DELETE, LOGIN, LOGOUT events with `old_values` and `new_values` (JSONB).

`v_recalled_product_sales`

All sales where the product `status` is `recalled` or `banned`. Any rows here are a compliance and patient safety issue requiring immediate action.

`v_prescription_compliance_violations`

(Defined in prescriptions domain, surfaced here for compliance reporting.) Schedule H1/X/NDPS drugs dispensed without a linked prescription.

Alert Rules

`ControlledSubstanceComplianceRule`

Fires CRITICAL if `v_prescription_compliance_violations` contains any rows.

`SupplierLicenseExpiryRule`

Fires CRITICAL/WARNING based on license expiry proximity.

Regulatory Obligations

- **Dispensing register**: `v_controlled_substance_register` must be made available for inspection on demand
- **Audit trail**: `v_audit_activity` satisfies the requirement for a change log of all entity modifications
- **Recalled products**: `v_recalled_product_sales` must be reviewed whenever a product recall is issued

Security Use Cases

- `v_inactive_user_accounts` — active accounts with no login for > 90 days should be suspended
- `v_audit_activity` filtered by `action = 'role_change'` detects privilege escalation events

- `v_audit_activity` filtered by `timestamp` outside business hours detects after-hours activity

Power BI Usage

`v_audit_activity` for compliance reporting and security reviews. `v_recalled_product_sales` for product recall impact assessment. `v_prescription_compliance_violations` for regulatory dashboard.

Domain: Supplier Management

Purpose

Monitor active suppliers, track drug license validity, and maintain a directory of supplier contacts and addresses. Drug license expiry is a critical compliance item — procurement from an unlicensed supplier is illegal under the Drugs and Cosmetics Act.

Status

Implemented as analytics views — data read from master DB via FDW.

Source Tables (via FDW)

- `master.suppliers` — core supplier master with `drug_license_no` and `license_expiry_date`
- `master.supplier_contacts` — named contacts per supplier
- `master.supplier_addresses` — billing, shipping, and registered addresses

Key Analytics Objects

`v_supplier_license_expiry`

Every active supplier with their license validity status: - `VALID` — license valid for more than 90 days - `WARNING` — license expires within 31–90 days - `CRITICAL` — license expires within 30 days - `EXPIRED` — license has already expired

Fields include primary contact name, phone, and email for immediate action.

`v_supplier_directory`

Active suppliers enriched with primary contact details and shipping address. Used for order dispatch and communication.

Alert Rules

`SupplierLicenseExpiryRule`

Reads `supplier_license_expiry` counts from the warehouse probe. Fires: - **CRITICAL** if any expired license exists - **CRITICAL** if any license expires within 30 days - **WARNING** if any license expires within 31–90 days

Procurement Performance

Full supplier performance KPIs are available in `v_supplier_performance` (procurement domain): - Fulfillment rate (received / created POs) - Average actual lead time vs declared `typical_lead_time_days` - Fill rate (units received / units ordered)

Power BI Usage

`v_supplier_license_expiry` for compliance monitoring. `v_supplier_directory` for contact lists. Join with `v_supplier_performance` on `supplier_id` for the full supplier scorecard.

Domain: Financial Analytics

Purpose

Provide accounts payable visibility (outstanding supplier payments), accounts receivable insight (customer collections), and payment method breakdowns for cash flow management.

Status

Implemented as analytics views — data read from master DB via FDW.

Source Tables (via FDW)

- `master.payments` — all payment records (customer and supplier)
- `master.purchase_orders` + `master.purchase_order_items` — PO values
- `master.sales` — invoice values

Key Analytics Objects

`v_accounts_payable`

Outstanding amounts owed to suppliers per purchase order. - Shows `po_total_amount`, `amount_paid`, `outstanding_amount`, and `age_days` - Filtered to POs with outstanding balance > 0 - Ordered by outstanding amount descending

`v_payment_method_breakdown`

Monthly payment volume by method (Cash, UPI, Credit Card, etc.) and payer type (customer/supplier).

`v_daily_collections`

Daily customer payment totals by method. Used for cash position monitoring and reconciliation.

GST / Tax Analytics

Revenue-side GST can be computed from `fact_sales_events.line_revenue` combined with `master.sale_items.tax_percentage_at_sale`. The `products.hsn_code` links to the applicable tax slab for GST filing.

Power BI Usage

- `v_accounts_payable` for AP aging report
- `v_payment_method_breakdown` for cash flow trend analysis
- `v_daily_collections` for daily cash reconciliation

Note on Completeness

This domain surfaces financial data for visibility only. The source of truth for all financial transactions remains the OLTP system. The analytics views are read-only and do not replace the accounting system.

Domain: Customer Analytics

Purpose

Understand customer purchasing behaviour, identify high-value customers, and track demographic patterns relevant to pharmaceutical dispensing (age groups, prescription frequency, government ID verification).

Status

Implemented as analytics views — data read from master DB via FDW.

Source Tables (via FDW)

- `master.customers` — customer master with PII fields
- `master.sales` — cross-reference for purchase history
- `master.prescriptions` — cross-reference for prescription history

Key Analytics Objects

`v_customer_summary`

Per-customer aggregation: - `lifetime_value` — total spend on completed sales - `total_purchases` — count of completed sales - `first_purchase_date` / `last_purchase_date` - `total_prescriptions` — number of

prescriptions linked to this customer - `age_years` — computed from `date_of_birth` - `id_verified` — whether `government_id` is populated (required for Schedule H1/X/NDPS)

Privacy Considerations

The `v_customer_summary` view exposes PII including `full_name`, `phone`, `email`, and `government_id`. Grant the `analytics_reader` role only to users who are authorised to view customer personal data.

The `password_hash` field from `users` is never exposed through any analytics view.

Segmentation

Customer types tracked in `fact_sales_events.customer_type`: - `RETAIL` — individual retail buyers - `HOSPITAL` — institutional bulk buyers - `DISTRIBUTOR` — wholesale distributors

Power BI can segment revenue and units by customer type without any joins.

Power BI Usage

`v_customer_summary` for customer lifetime value and loyalty analysis. Cross-join with `v_revenue_summary` on `customer_type` for segment-level revenue breakdown.

Domain: Operations & Staff Performance

Purpose

Monitor staff productivity, detect operational anomalies (excessive adjustments, after-hours activity), and maintain account hygiene (inactive accounts).

Status

Implemented as analytics views — data read from master DB via FDW.

Source Tables (via FDW)

- `master.users` — staff accounts with roles and last login
- `master.sales` — cross-reference via `sales.user_id`
- `master.stock_adjustments` — cross-reference via `adjusted_by`
- `master.goods_receipts` — cross-reference via `received_by`
- `master.audit_logs` — activity timeline per user

Key Analytics Objects

`v_staff_performance`

Per-user summary of all operational activity: - `completed_sales` — count of completed sales processed - `total_revenue` — gross revenue from completed sales - `returns_processed` — count of returned sales - `stock_adjustments_made` — number of manual stock adjustments - `grns_received` — number of GRNs processed - `last_login` — most recent login timestamp

`v_inactive_user_accounts`

Active accounts with no login for more than 90 days. These are a security risk and should be suspended. Fields: `user_id`, `username`, `role`, `last_login`, `days_since_login`

Operational Anomaly Patterns

Using `v_staff_performance`: - High `stock_adjustments_made` relative to peers → investigate for shrinkage or errors - High `returns_processed` rate → investigate customer complaint patterns or fraud

Using `v_audit_activity` (compliance domain): - Filter `user_id` + `timestamp` outside 08:00–20:00 for after-hours activity review - Filter `entity = 'users'` + `action = 'UPDATE'` for role change detection

Power BI Usage

`v_staff_performance` for operations management dashboards. `v_inactive_user_accounts` for IT security reporting. Set up automated alerts using `v_inactive_user_accounts` row count.

Privacy Note

Staff performance data is sensitive HR information. Access should be restricted to managers and administrators. Apply PostgreSQL row-level security if per-manager scoping is needed in the future.

Domain: Prescription Analytics & Compliance

Purpose

Track prescription dispensing for regulatory compliance. Indian drug laws require controlled substances (Schedule H1, X, NDPS) to be dispensed only with a valid prescription. This domain provides the audit views and compliance checks required by law.

Status

Implemented as analytics views — no separate Kafka pipeline. Data is read directly from the master database via FDW.

Source Tables (via FDW)

- `master.prescriptions` — prescription headers (doctor, patient, date)
- `master.prescription_items` — prescribed drugs with dosage
- `master.sales` — cross-referenced via `sales.prescription_id`
- `master.products` — `drug_schedule` field (OTC, Schedule H, H1, X, NDPS)
- `master.customers` — customer details including `government_id`

Key Analytics Objects

`v_prescription_fill_rate`

Per-prescription summary: how many prescribed products resulted in a completed sale.

`v_controlled_substance_register`

Legally required register. All Schedule H1, X, and NDPS drug dispensing records including: - Patient name and government ID - Drug name, schedule, and quantity - Prescribing doctor and prescription date - Whether a prescription was linked (`missing_prescription` flag)

`v_prescription_compliance_violations`

Subset of `v_controlled_substance_register` where `missing_prescription = TRUE`. Any rows here represent active regulatory violations.

`v_doctor_prescribing_patterns`

Drug-level prescribing volume per doctor for utilisation review.

Alert Rule

`ControlledSubstanceComplianceRule` — fires CRITICAL if any violations exist in `v_prescription_compliance_violations`. Requires the warehouse DB to be reachable.

Regulatory Context

Under the Drugs and Cosmetics Act (India): - **Schedule H**: Prescription required, recorded in dispensing register - **Schedule H1**: Prescription retained, patient identity verified - **Schedule X**: Prescription in triplicate, government forms may be required - **NDPS**: Narcotic Drugs and Psychotropic Substances — most restricted, strict register required

Power BI Usage

Connect to `v_controlled_substance_register` for the regulatory dispensing register. Connect to `v_prescription_compliance_violations` for compliance monitoring. No joins needed.

No Pipeline Required

Prescription data is operational (entered at point of sale in the OLTP system). Analytics reads it directly via FDW — no Kafka events or Spark jobs needed.

Functional Requirements Checklist

Assessment date: 2026-06-06

Status legend

- **Complete** — implemented and tested
- **Partial** — implemented structurally; integration tests against a live DB are not yet automated
- **Not started** — no meaningful implementation

Summary

Domain	Pipeline	Analytics Views	Alerts	Tests
Inventory	Complete	Complete	Complete	Unit tests pass
Procurement	Complete	Complete	Freshness alerts	Unit tests pass
Sales	Complete	Complete	Freshness alerts	Unit tests pass
Prescriptions	FDW views	Complete	Compliance alert	—
Supplier	FDW views	Complete	License expiry alert	—
Financial	FDW views	Complete	—	—
Customer	FDW views	Complete	—	—
Audit/Compliance	FDW views	Complete	—	—
Staff Performance	FDW views	Complete	—	—

Automated test count: **82 unit tests passing** across 8 test files. Integration tests (requiring a live PostgreSQL connection) are not yet automated.

Functional Checklist

Area	Requirement	Status	Notes
Platform foundation	Single CLI entrypoint for all operations	Complete	`python -m medwarehouse <group> <cmd>`
Configuration	All settings from MW_* env vars	Complete	`config.py`, weak credential detection in non-local envs
Security	API key auth on both Flask services	Complete	`platform/auth.py`, MW_WEBAPP_API_KEY + MW_PLATFORM_API_KEY
Inventory contracts	Event schema + validation	Complete	`contracts/inventory.py`, shared utils in `contracts/_utils.py`
Procurement contracts	Event schema + validation (4 types)	Complete	`contracts/procurement.py`
Sales contracts	Event schema + validation (2 types)	Complete	`contracts/sales.py`
Inventory producer	Deterministic sample events to Kafka	Complete	All 4 event types
Procurement producer	Full lifecycle events to Kafka	Complete	All 4 event types: CREATED→APPROVED→RECEIVED, with cancellations
Sales producer	Sales + cancellation events to Kafka	Complete	Both types, all 3 customer types, realistic price tiers
Bronze ingestion	Kafka → Parquet (all 3 domains)	Complete	Shared `_bronze.py` utility
Silver transformation	Validate, deduplicate, quarantine (all 3 domains)	Complete	Shared `_silver.py` deduplication logic
Warehouse staging	Silver → PostgreSQL staging (all 3 domains)	Complete	Shared `_stage.py` JDBC writer
Warehouse bootstrap	Schemas, roles, FDW, dims, facts, views, grants	Complete	`sql/warehouse/01-10_*.sql` applied in order
FDW integration	Master DB read-only via FDW (16 tables)	Complete	User mappings for all 4 roles
Dimensions	SCD Type-2 product, supplier, warehouse, customer	Complete	`sql/warehouse/03_inventory_dimensions.sql`, `09_sales_warehouse.sql`
Inventory fact	Append-only partitioned fact table	Complete	`fact_inventory_events`
Procurement fact	Append-only partitioned fact table	Complete	`fact_procurement_events`
Sales fact	Append-only partitioned fact table with line_revenue	Complete	`fact_sales_events`
Inventory semantic views	v_inventory_balance, v_inventory_snapshot	Complete	
Procurement semantic views	v_po_lifecycle, v_supplier_performance, v_po_aging	Complete	
Sales semantic views	v_revenue_summary, v_sales_velocity, v_returns_analysis	Complete	
Reorder automation	Snapshot table + policy-based PO creation	Complete	`fact_inventory_balance`, `reorder_policies`, `check_reorder_thresholds()`
Purchase order management	DRAFT → APPROVED → SENT → RECEIVED lifecycle	Complete	`pending_purchase_orders` + webapp API

Area	Requirement	Status	Notes
Prescription compliance views	v_controlled_substance_register, violations	Complete	FDW-backed
Supplier license monitoring	v_supplier_license_expiry, CRITICAL/WARNING/EXPIRED	Complete	FDW-backed + alert rule
Financial analytics	v_accounts_payable, v_daily_collections	Complete	FDW-backed
Customer analytics	v_customer_summary	Complete	FDW-backed
Audit analytics	v_audit_activity, v_recalled_product_sales	Complete	FDW-backed
Staff performance	v_staff_performance, v_inactive_user_accounts	Complete	FDW-backed
Quality checks	Data integrity assertions (all 3 domains)	Complete	7 + 4 + 5 checks across inventory/procurement/sales
Inventory Airflow DAG	8-task orchestrated pipeline	Complete	`inventory_gold_pipeline.py`
Procurement Airflow DAG	6-task orchestrated pipeline	Complete	`procurement_gold_pipeline.py`
Sales Airflow DAG	6-task orchestrated pipeline	Complete	`sales_gold_pipeline.py`
Monitoring platform	Status dashboard, job runner, alerts UI	Complete	`platform/` at port 8787
Monitoring: procurement/sales probes	Pipeline freshness + artifact tracking	Complete	`PipelineProbe`, `ArtifactProbe` cover all 3 domains
Alert engine	Freshness, volume, compliance, infrastructure alerts	Complete	14 alert rules across inventory, procurement, sales, compliance
Operator webapp REST API	Stock entry, inventory lookup, reorder, PO management	Complete	`medwarehouse_webapp/` at port 8080
Operator webapp frontend	HTML/CSS/JS UI	**Not started**	REST API is complete; frontend to be built separately
Shell scripts	Local flow scripts for all 3 domains	Complete	`scripts/run_*_flow.sh`
Documentation	architecture, data-dict, API reference, deployment, security, 9 domain docs	Complete	`docs/`
Unit tests	Contract validation, config, platform, auth, warehouse functions, webapp services	Complete	82 tests across 8 files
Integration tests	Live DB stored procedure tests	**Not started**	Requires `MW_INTEGRATION_TEST=1` and live PostgreSQL
CI/CD	Automated test pipeline	**Not started**	No GitHub Actions configured

---

Remaining Work

1. **Operator webapp frontend** — HTML/CSS/JS for the 6 operator pages. REST API is complete and documented in `docs/api-reference.md`.
2. **Integration tests** — Tests that exercise stored procedures against a live PostgreSQL instance. Framework is in place (`MW_INTEGRATION_TEST=1`).
3. **CI/CD** — GitHub Actions or equivalent to run tests on every push.

4. **Power BI connection** — Step-by-step guide for connecting Power BI to `analytics_reader` and importing recommended views.

Control Panel Testing Plan

Assessment date: 2026-04-27 UTC (source paths updated 2026-06-06)

This page defines how to test the Medical Warehouse control panel based on the current implementation in `src/medwarehouse/platform/`, the UI template, the JSON endpoints, and the existing automated tests.

Scope

The control panel currently includes:

- a Flask web UI served by `python -m medwarehouse platform serve`
- JSON status and control APIs
- infrastructure start/stop actions
- job start/stop actions
- status reporting for environment, warehouse, orchestration, coverage, roles, and local data artifacts

Primary source files:

- `src/medwarehouse/platform/api/app.py` — Flask application factory
- `src/medwarehouse/platform/api/routes.py` — API + page route blueprints
- `src/medwarehouse/platform/control/` — Job runner, infra actions, `ProcessSupervisor`, job catalog
- `src/medwarehouse/platform/services/status.py` — `StatusService` (TTL cache + probes)
- `src/medwarehouse/platform/services/alerts/` — Alert engine and rule modules
- `src/medwarehouse/platform/probes/` — `InfraProbe`, `WarehouseProbe`, `PipelineProbe`, etc.
- `src/medwarehouse/platform/ui/templates/` — Jinja2 templates (base.html + page templates)
- `src/medwarehouse/platform/control/catalog.py` — `JOB_SPECS` catalog
- `tests/test_platform.py`

Test objectives

1. Confirm that the control panel loads and exposes the expected status information.
2. Confirm that UI controls and JSON APIs trigger the correct backend actions.
3. Confirm that the control panel behaves safely when infrastructure, warehouse, or jobs are unavailable.
4. Confirm that the control panel remains usable, understandable, and responsive in normal local-development conditions.

Test environments

Minimum local setup

```
export PYTHONPATH=src
set -a && source .env && set +a
python -m medwarehouse platform serve --host 127.0.0.1 --port 8787
```

Recommended environment variants

- Variant A: Docker available, all services stopped
- Variant B: Docker available, core services running
- Variant C: Docker unavailable or intentionally mocked as unavailable
- Variant D: Analytics DB reachable
- Variant E: Analytics DB unreachable or wrong credentials
- Variant F: Jobs idle
- Variant G: One long-running job active, such as `inventory_bronze`

Functional requirements

ID	Requirement	Expected behavior	Test type	Current coverage
FR-01	The control panel root page shall render successfully	`GET /` returns HTTP 200 and shows the main control-plane sections	Automated + manual	Partial
FR-02	A health endpoint shall be available	`GET /health` returns a simple healthy response	Manual + automated	Partial
FR-03	A JSON status endpoint shall expose snapshot, jobs, and infra state	`GET /api/status` returns structured JSON with those top-level keys	Manual + automated	Partial
FR-04	The page shall display infrastructure overall status and per-service status	UI shows overall state and service rows from Docker inspection	Manual	Gap
FR-05	The page shall allow starting infrastructure	`POST /api/infra/start` and form flow trigger `start_services.sh` through the infra controller	Manual + automated	Partial
FR-06	The page shall allow stopping infrastructure	`POST /api/infra/stop` and form flow trigger `stop_services.sh` through the infra controller	Manual + automated	Partial
FR-07	The page shall display environment metadata	UI shows environment name, `.env` presence, Kafka target, DB targets, and project root	Automated + manual	Partial
FR-08	The page shall display domain coverage information	UI shows inventory, procurement, and sales coverage cards	Manual	Gap
FR-09	The page shall display configured warehouse role names and detected roles	UI shows configured role names and warehouse-detected roles or `unavailable`	Manual	Gap
FR-10	The page shall display orchestration metadata	UI shows DAG id, DAG path, and DAG existence status	Partial automated + manual	Partial
FR-11	The page shall display warehouse reachability and key row counts	UI shows reachability, target JDBC URL, snapshot row count, fact row count, staging row count, and schemas	Manual	Gap
FR-12	The page shall display Bronze, Silver, and Quarantine artifact summaries	UI shows existence, file count, updated time, and path for each artifact	Manual	Gap

ID	Requirement	Expected behavior	Test type	Current coverage
FR-13	The page shall list all configured pipeline jobs	All entries from `JOB_SPECS` appear in the UI and status payload	Manual + automated	Gap
FR-14	The page shall allow starting configured jobs	`POST /api/jobs/<job_id>/start` starts the requested command and returns updated job state	Manual + automated	Partial
FR-15	The page shall allow stopping long-running jobs	`POST /api/jobs/<job_id>/stop` stops a running long job and returns updated job state	Manual + automated	Partial
FR-16	The page shall prevent duplicate starts of the same running job	Repeated start requests for an active job return `ok=false` with a clear message	Manual + automated	Gap
FR-17	The page shall reject stop requests for jobs that are not running	Stop request returns `ok=false` with a clear message	Manual + automated	Gap
FR-18	The page shall reject unknown job ids gracefully	Unknown job requests return a non-crashing JSON response with `ok=false`	Manual + automated	Gap
FR-19	The page shall reject unknown infra actions gracefully	Unknown infra action returns a non-crashing JSON response with `ok=false`	Manual + automated	Complete — `_ALLOWED_INFRA_ACTIONS` whitelist enforced in `api/routes.py`
FR-20	The page shall show recent job logs and infra logs	Latest log lines appear in the UI after command execution	Manual	Gap
FR-21	The refresh button shall reload the page	Clicking refresh reloads the current page without breaking state	Manual	Gap
FR-22	The button-based UI controls shall call the correct JSON endpoints	Clicking start/stop/refresh buttons performs the intended action and reload flow	Manual	Gap
FR-23	Warehouse connection failures shall be surfaced with actionable hints	UI shows an error note rather than crashing when DB is unreachable or auth fails	Manual	Partial
FR-24	Missing Docker or Compose availability shall be surfaced clearly	Infrastructure section shows `unavailable` and explanatory text	Automated + manual	Partial

Functional test checklist

PAGE AND API SMOKE

1. Start the control panel.

2. Open `http://127.0.0.1:8787/`.
3. Confirm the page renders with sections for Infrastructure, Environment, Coverage, Roles, Orchestration, Warehouse, Artifacts, and Pipeline Controls.
4. Call `GET /health` and confirm a healthy JSON response.
5. Call `GET /api/status` and confirm the JSON includes `snapshot`, `jobs`, and `infra`.

INFRASTRUCTURE CONTROLS

1. With Docker available and services stopped, click `Start Services`.
2. Confirm the infra action enters `running`, then reaches `succeeded` or a clear `failed` state.
3. Confirm recent infra logs appear in the infrastructure log panel.
4. Confirm per-service badges update after refresh.
5. Repeat with `Stop Services`.
6. Attempt a second infra action while one is already running and confirm it is rejected cleanly.

JOB CONTROLS

1. Start a short job such as `Inventory Producer`.
2. Confirm the job badge changes from `idle` to `running`, then to `succeeded` or `failed`.
3. Confirm PID, timestamps, and logs update.
4. Start a long-running job such as `Inventory Bronze Stream`.
5. Confirm the `Stop` button is available for long-running jobs.
6. Stop the job and confirm the state transitions through `stopping` to `stopped`.
7. Try starting the same long-running job twice and confirm the second request is rejected.
8. Try stopping an idle job and confirm the request is rejected without crashing the page.

STATUS CONTENT

1. With `.env` present, confirm the environment section shows `yes`.
2. Temporarily test with `.env` absent or a mocked project root and confirm the badge changes appropriately.
3. With services stopped, confirm warehouse reachability is `no` and an explanatory note is displayed.
4. With services running and warehouse bootstrapped, confirm counts and schemas populate.
5. Confirm Bronze, Silver, and Quarantine artifact summaries reflect actual filesystem state.
6. Confirm coverage cards correctly show:
   - inventory as end-to-end implemented
   - procurement as end-to-end implemented
   - sales as end-to-end implemented

FAILURE HANDLING

1. Run with Docker unavailable and confirm the infrastructure section shows `unavailable`.
2. Run with wrong analytics DB credentials and confirm warehouse failure is shown as a note, not a server error.
3. Call `POST /api/jobs/unknown/start` and confirm a safe JSON error response.
4. Call `POST /api/infra/unknown` and confirm a safe JSON error response.

Non-functional requirements

ID	Requirement	Target expectation	How to test	Current confidence
NFR-01	Availability	Control panel should start locally with a single command when Python dependencies are installed	Start server from CLI and open `/` and `/health`	Medium
NFR-02	Fault tolerance	Backend should return controlled error states instead of crashing when Docker or warehouse is unavailable	Test stopped infra, wrong DB credentials, unknown job ids, unknown infra actions	Medium
NFR-03	Responsiveness	Page load and JSON status should feel fast for a local setup, ideally within about 1 to 2 seconds for typical local data volumes	Measure browser load time and `curl` timing for `/` and `/api/status`	Low
NFR-04	Concurrency safety	Repeated user clicks should not create duplicate job starts or overlapping infra actions	Attempt repeated starts/stops rapidly and verify lock-based rejection	Medium
NFR-05	Observability	Recent logs, PID, timestamps, and status badges should provide enough context to debug local runs	Execute jobs and review logs shown in UI	Medium
NFR-06	Usability	UI labels and statuses should be understandable to an operator without reading source code	Manual UX review with stopped and running states	Medium
NFR-07	Accessibility	Basic keyboard usage, readable contrast, and responsive layout should work on desktop and mobile widths	Manual keyboard and viewport testing	Low
NFR-08	Security posture	UI should avoid arbitrary command execution; only predefined job specs and fixed infra scripts should be invokable	Review job catalog and request surface; confirm unknown ids are rejected	Medium
NFR-09	Data accuracy	Status values shown in the UI	Cross-check UI values with CLI commands and local files	Medium

ID	Requirement	Target expectation	How to test	Current confidence
		should match the underlying Docker, filesystem, and database state at refresh time		
NFR-10	Maintainability	Control panel behavior should remain covered by automated tests as routes and sections evolve	Review and extend `tests/test_platform.py`	Low to medium
NFR-11	Portability	The control panel should work across local environments where Python, Docker, and Compose path variants differ	Test with `docker compose` and `docker-compose` availability	Medium
NFR-12	Resource usage	Idle control-panel operation should remain lightweight in CPU and memory	Observe process usage while idle and while refreshing status	Low

Non-functional test checklist

Performance and responsiveness

1. Measure `GET /` and `GET /api/status` response times with services stopped.
2. Repeat with services running and warehouse reachable.
3. Confirm the page remains usable when artifact directories contain many parquet files.

Concurrency and stability

1. Double-click `Start` on a long-running job.
2. Rapidly issue repeated `POST /api/jobs/<job_id>/start` requests.
3. Rapidly issue repeated `POST /api/infra/start` requests.
4. Confirm only one active process exists for each protected action.

Usability and layout

1. Verify the page at desktop width.
2. Verify the page below `900px` width and confirm the grid collapses cleanly into one column.
3. Verify logs, code blocks, and long file paths wrap without breaking layout.
4. Check that status colors remain distinguishable.

Security and safety

1. Confirm only known jobs from `JOB_SPECS` can be started.
2. Confirm only `start` and `stop` are accepted as infra actions.
3. Confirm command strings shown in the UI are descriptive and match the actual backend action.
4. Review whether any sensitive values are unintentionally exposed in the UI, especially full connection targets or auth-related error text.

Existing automated coverage

Current tests in `tests/test_platform.py` cover:

- snapshot structure exists
- index page renders

- `GET /health`
- `GET /api/status`
- `POST /api/jobs/<job_id>/start`
- `POST /api/jobs/<job_id>/stop`
- `POST /api/infra/<action>`
- CLI `platform serve` calls the platform server
- Docker-missing path returns an unavailable status
- control-plane service duplicate-start rejection for jobs
- control-plane service idle-stop rejection for jobs
- control-plane service unknown infra-action rejection

Current automated coverage does not yet cover:

- unknown job or infra action handling
- duplicate-start handling through the public API
- stop-on-idle handling through the public API
- UI rendering of dynamic job lists
- responsive UI behavior
- live warehouse status behavior
- actual subprocess launch/stop lifecycle
- persistence and restart recovery behavior

Recommended automation backlog

1. Add API tests for unknown job ids and unknown infra actions.
2. Add API-level tests for duplicate job starts and stop-on-idle behavior.
3. Add direct tests for actual subprocess launch, log capture, and stop escalation in the shared supervisor/runtime.
4. Add persistence and restart-recovery tests for the SQLite-backed control-plane store.
5. Add tests that assert the index includes key sections and at least one pipeline job card.
6. Add tests for warehouse error-hint formatting when authentication fails.
7. Add tests for live warehouse status behavior with mocked database cursors and partial failures.
8. Add browser-level checks for responsive layout and operator feedback on failed actions.

Suggested release gate for the control panel

The control panel should be considered test-ready for routine local use when all of the following are true:

1. Functional smoke tests pass for `/`, `/health`, `/api/status`, job start, job stop, infra start, and infra stop.
2. The page renders correctly with services both stopped and running.
3. Unknown actions are rejected safely.
4. Long-running jobs can be stopped from the UI.
5. Basic responsiveness and mobile-layout checks pass.
6. Automated tests cover all public routes and the main controller edge cases.

Overall assessment

- Current implementation maturity: good local-operator prototype
- Current automated test maturity: early
- Biggest functional testing gap: API route behavior and controller edge cases
- Biggest non-functional testing gap: responsiveness, usability, and concurrency verification